



**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge–Fog–Cloud**

**Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra**

Universidad de Los Andes
Departamento de Ingeniería de sistemas y computación
Bogotá, Colombia
2026

**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge–Fog–Cloud.**

**Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra**

University de Los Andes
Presentado en cumplimiento parcial de los requisitos para el grado de:
System and Computer engineer

Director:
Prof. Carlos Andrés Lozano Garzón, PhD
Trabajo de grado defendido en:
Mayo 24, 2026 - 12:00 pm

Universidad de Los Andes
Departamento de Ingeniería de Sistemas y Computación
Bogotá, Colombia
2026

Agradecimientos

Quiero expresar mi más sincera gratitud a todas las personas que hicieron posible este logro.

A mi familia, quienes han sido mi ancla y mi inspiración en cada paso de este camino. Su amor incondicional y apoyo constante me han dado la fortaleza para superar cualquier desafío.

Al profesor Paulo de Oliveria, mi asesor, quien no solo compartió generosamente su conocimiento y experiencia, sino que también fue un guía incansable durante este proyecto. Su paciencia, dedicación y constante disposición para resolver dudas fueron esenciales para enfrentar los retos que surgieron. Sus valiosos comentarios y orientaciones me permitieron aprender y crecer, tanto a nivel académico como personal. Este proyecto no habría sido lo mismo sin su acompañamiento e inspiración.

A la Universidad de los Andes, mi alma máter, por ser un espacio de excelencia académica que me permitió desarrollar mis habilidades y consolidar mi formación profesional. A sus profesores, administrativos y recursos de investigación, les agradezco por fomentar un ambiente de aprendizaje integral, colaboración y crecimiento personal. Haber formado parte de esta comunidad académica ha sido un privilegio invaluable que marcará para siempre mi trayectoria.

Y, con un cariño especial, a mis amigos. Gracias por convertir este recorrido en una aventura inolvidable. A Sebastián Moya, por su apoyo clave y su disposición.

A todos ustedes, mi eterno agradecimiento. Este logro también es suyo.

Resumen

Este documento presenta el diseño, implementación y evaluación de un sistema de detección de intrusiones (IDS) para redes IoT/MQTT desplegado en una arquitectura jerárquica Edge-Fog-Cloud. El sistema combina modelos livianos de *machine learning* ejecutados en microcontroladores ESP32-S3, aprendizaje federado jerárquico (HFL) coordinado desde gateways Raspberry Pi 4 y un servidor PC, y cifrado ligero **ASCON-128** para proteger el ciclo de aprendizaje. La investigación evalúa cuatro dimensiones: viabilidad de TinyML en el borde, reducción de transferencia de datos mediante HFL, overhead criptográfico de ASCON y comparación entre variantes de arquitectura y modelo.

Se diseñó un perceptrón multicapa (MLP) compacto de 1 139 parámetros (≈ 4.5 KB) con arquitectura $13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$, capaz de clasificar flujos de red en tres categorías (**normal**, **mqtt_bruteforce**, **scan_A**) utilizando 13 características extraídas de flujos. El sistema federado comparte únicamente las dos últimas capas del modelo (163 parámetros) y fue evaluado en las ramas **hfl_v7-RN**, **hfl_v7-no-ascon** y **hfl_v7-CNN**, incluyendo modo estándar y modo FOG.

Como línea base, se entrenaron modelos clásicos centralizados, alcanzando hasta 99.94 % de accuracy con Random Forest, aunque con requerimientos de memoria incompatibles con microcontroladores. El notebook general de entrenamiento comparó además MLP/RN, CNN-1D, Residual-MLP y Tiny Transformer; el MLP/RN obtuvo el mejor balance entre precisión offline ($\approx 90.6\%$), tamaño e implementación en ESP32. La cuantificación del overhead de ASCON-128 mostró costo temporal bajo frente a la duración de las rondas federadas y un incremento moderado del tamaño de los mensajes, sin degradación observable de convergencia.

Los resultados validan que la combinación de TinyML, aprendizaje federado y cifrado ligero constituye una solución viable y segura para detección de intrusiones en el borde de redes IoT, ofreciendo un compromiso efectivo entre precisión, privacidad, seguridad y eficiencia computacional.

Keywords: Detección de Intrusiones, IoT, TinyML, Aprendizaje Federado Jerárquico, ASCON-128, Criptografía Ligera, Edge-Fog-Cloud, ESP32-S3, Raspberry Pi, CNN-1D, MLP, Seguridad de Redes.

Tabla de Contenido

List of Figures	xi
-----------------	----

List of Tables	xiii
----------------	------

1. Introducción	1
1.1. Contexto	1
1.2. Problemática	2
1.3. Justificación	2
1.4. Objetivos	3
1.4.1. Objetivo General	3
1.4.2. Objetivos Específicos	3
2. Marco Teórico	4
2.1. Marco Conceptual	4
2.1.1. IDS en redes IoT/IIoT	4
2.1.2. Representación por flujos	4
2.1.3. TinyML y modelos compactos	4
2.1.4. Aprendizaje federado jerárquico	5
2.1.5. Cifrado ligero y ASCON-128	5
2.1.6. Agregación FOG	6
2.2. Estado del Arte	6
2.2.1. IDS y TinyML para IoT/IIoT	6
2.2.2. Aprendizaje federado para seguridad IoT	7
2.2.3. Criptografía ligera y autenticación	7
2.2.4. Datasets y representación del tráfico	7
2.3. Síntesis de la Brecha	8
3. Metodología	10
3.1. Modelo de Amenazas y Suposiciones	10
3.2. Arquitectura del Sistema	11
3.2.1. Capa Edge: Nodos ESP32-S3	11
3.2.2. Capa Fog: Gateway Raspberry Pi 4	11
3.2.3. Capa Cloud: Servidor PC	12
3.2.4. Modos de operación	12

3.3.	Conjunto de Datos	12
3.4.	Modelo de Aprendizaje: MLP Multiclase	13
3.5.	Modelos livianos evaluados	15
3.6.	Protocolo de Aprendizaje Federado Jerárquico	15
3.7.	Integración de Cifrado Ligero ASCON-128	18
3.7.1.	Parámetros Criptográficos	18
3.7.2.	Canales Cifrados	18
3.7.3.	Métricas de Overhead	19
3.8.	Variantes implementadas en <code>hf1_v7</code>	19
3.9.	Modelos Clásicos de ML como Línea Base	19
3.10.	Protocolo Experimental	20
3.10.1.	Escenario 1: Sistema HFL Completo con ASCON	20
3.10.2.	Escenario 2: Sistema HFL sin ASCON (Baseline)	21
3.10.3.	Escenario 3: Sistema HFL con CNN-1D	21
3.10.4.	Escenario 4: Estrategia FOG	21
3.10.5.	Escenario 5: Modelos Clásicos Centralizados	21
3.10.6.	Métricas de Evaluación	21
4.	Resultados	22
4.1.	Desempeño de Modelos Clásicos (Línea Base Centralizada)	22
4.1.1.	Viabilidad de Despliegue en ESP32	23
4.2.	Comparación de Modelos Livianos	23
4.3.	Aprendizaje Federado Jerárquico: Convergencia y Precisión	24
4.3.1.	Convergencia del Modelo Federado	24
4.3.2.	Evolución temporal de accuracy, loss y pesos	24
4.3.3.	Comparación: Federado vs. Centralizado	26
4.3.4.	Comparación entre variantes <code>hf1_v7</code>	26
4.3.5.	Comparación con la literatura	28
4.3.6.	Limitaciones del estudio	29
4.4.	Overhead del Cifrado ASCON-128	30
4.4.1.	Overhead Temporal	31
4.4.2.	Overhead de Comunicación	32
4.4.3.	Resumen: Impacto de ASCON en el Sistema HFL	33
4.5.	Análisis de las Preguntas de Investigación	33
4.5.1.	PI-1: ¿Puede TinyML detectar ataques de red en dispositivos IoT con precisión operativa?	33
4.5.2.	PI-2: ¿Reduce el aprendizaje federado el intercambio de datos sensibles sin afectar la precisión del IDS?	34
4.5.3.	PI-3: ¿Cuál es el overhead del cifrado ASCON en la comunicación de modelos federados?	34

4.5.4. PI-4: ¿Qué aporta comparar MLP/RN, CNN-1D, baseline sin ASCON y FOG?	35
5. Conclusiones y Trabajos Futuros	36
5.1. Conclusiones	36
5.1.1. Sobre la viabilidad de TinyML para IDS en IoT	36
5.1.2. Sobre el aprendizaje federado y la privacidad	37
5.1.3. Sobre el overhead del cifrado ASCON-128	37
5.1.4. Sobre la arquitectura integrada	38
5.2. Amenazas no cubiertas y trabajos futuros	38
Bibliografía	40
A. Anexos	45
A.1. Anexo A: Arquitectura del Sistema HFL	45
A.2. Anexo B: Archivos del Sistema	47
A.3. Anexo C: Repositorio del Proyecto	47
A.4. Anexo D: Resumen de literatura clave incorporada	48

Lista of Figuras

3-1.	Modos de operación: (a) estándar con agregación directa PC–gateways y (b) FOG con agregación jerárquica mediante un gateway líder.	12
3-2.	Arquitectura del modelo MLP multiclasa. Las capas ocultas 3 y de salida (recuadro punteado) conforman las <i>capas federadas</i> (W_3 , b_3 , W_4 , b_4), cuyos 163 parámetros son los únicos intercambiados durante el aprendizaje federado.	14
3-3.	Diagrama de secuencia del protocolo HFL v7 bidireccional. Los actores PC (Cloud), Raspberry Pi (Fog) y ESP32-S3 (Edge) intercambian mensajes en tres fases: despliegue del modelo global, recolección de features con entrenamiento local y agregación FedAvg. Todos los canales MQTT/HTTP se protegen con ASCON-128 cuando la variante está habilitada; en modo FOG se añaden los topics <code>fog/weights</code> y <code>fog/global_model</code> entre Raspberry Pi peer y leader.	17
4-1.	Accuracy y loss globales promedio por ronda (1–20) para las 8 variantes HFL v7. Generado por <code>sre_completo_analysis.py</code> a partir de los CSV de <code>global_weights_history_*</code>	25
4-2.	Magnitud promedio de los cuatro pesos federados (<code>w3_mag</code> , <code>w4_normal</code> , <code>w4_brute</code> , <code>w4_scan</code>) por variante a lo largo de las rondas 1–20.	25
4-3.	Duración promedio por ronda federada para las 8 variantes (rondas 1–20).	28
4-4.	Desbalance de <i>accuracy</i> entre gateways ($ acc_A - acc_B $) por ronda y por variante.	28
4-5.	Latencia p95 (ms) promedio por variante y por canal/operación. Las variantes <code>*PLAIN*</code> solo muestran la latencia de serialización JSON (sub-milisegundo); las <code>*ASCON*</code> incluyen el costo de cifrado/descifrado autenticado.	30
4-6.	Tamaño promedio de plaintext (izquierda) y overhead promedio sobre plaintext (derecha) por variante y canal. El overhead de las variantes PLAIN es 0 por construcción.	31

-
- A-1.** Arquitectura física del sistema HFL v7. La capa Cloud (PC) coordina las rondas con FastAPI y FedAvg ponderado. La capa Fog (Raspberry Pi) actúa como broker MQTT, etiquetador heurístico y entrenador local; en el modo FOG, una segunda Raspberry Pi participa como *peer* y se comunica con el *leader* mediante los topics `fog/weights` y `fog/global_model` (línea discontinua). La capa Edge (ESP32-S3) ejecuta inferencia TinyML sobre 13 features de flujo y publica datos al gateway. ASCON-128 cubre todos los canales en variantes `*_ASCON*`; en variantes `*_PLAIN*` los topics y endpoints usan sufijo `_plain` con JSON sin cifrar. 45

Lista de Tablas

2-1. Relación entre literatura y decisiones de diseño.	8
3-1. Composición del dataset de flujos de red.	13
3-2. Vector de 13 features extraídas por flujo.	13
3-3. Distribución de parámetros del modelo MLP.	14
3-4. Modelos livianos entrenados y artefactos exportados.	15
3-5. Canales de comunicación protegidos con ASCON-128.	18
3-6. Variantes de arquitectura evaluadas.	19
4-1. Desempeño de modelos clásicos de ML (entrenamiento centralizado).	22
4-2. Estimación de huella de memoria para despliegue en ESP32.	23
4-3. Comparación offline de modelos livianos sobre las 13 features.	23
4-4. Métricas de convergencia del sistema HFL.	24
4-5. Comparación de precisión: entrenamiento federado vs. centralizado.	26
4-6. Resumen comparativo de las 8 variantes HFL v7 evaluadas. GW p95 : latencia p95 de descifrado en gateway (ESP32→RPi). Srv dec p95 : latencia p95 de descifrado en servidor (RPi→PC). Overhead : bytes adicionales promedio sobre el plaintext (0 en variantes PLAIN).	27
4-7. Comparación cualitativa con trabajos relacionados.	29
4-8. Latencia p95 (ms) promedio de cifrado/descifrado por variante, derivada de transport_sli_summary.csv en Analisis de Modelos/Completo/.	31
4-9. Overhead promedio de tamaño por variante ASCON. Plaintext : bytes promedio de payload sin cifrar (pt_bytes). Overhead : bytes adicionales del envelope ASCON+Base64 (overhead_bytes). Ratio : porcentaje sobre el plaintext (overhead_bytes / pt_bytes). Las variantes PLAIN se omiten porque el overhead es 0 por construcción.	32
4-10. Resumen del overhead total de ASCON-128 por ronda federada, agregando los promedios de las 4 variantes ASCON y las 4 variantes PLAIN.	33
A-1. Archivos principales del sistema HFL v7, agrupados por capa de despliegue.	47
A-2. Literatura clave y aporte concreto a la tesis.	48

1. Introducción

1.1. Contexto

La adopción de redes *Internet of Things* (IoT) e *Industrial IoT* (IIoT) ha aumentado la superficie de ataque de hogares, laboratorios, plantas industriales y sistemas de monitoreo conectados. Estos entornos combinan dispositivos heterogéneos, protocolos livianos como MQTT, conectividad intermitente y capacidades limitadas de cómputo, lo que dificulta desplegar mecanismos de detección convencionales basados en procesamiento centralizado [1, 2]. Bajo marcos de gestión de riesgo como el *NIST Cybersecurity Framework*, la detección temprana y el monitoreo continuo son capacidades fundamentales para reducir el tiempo de respuesta ante incidentes [3].

Los sistemas de detección de intrusiones de red (NIDS) basados en aprendizaje automático han mostrado altos niveles de precisión en condiciones de laboratorio. Sin embargo, muchos de estos resultados se obtienen bajo supuestos que no siempre se cumplen en IoT: centralización de datos, disponibilidad de servidores con alta memoria, ausencia de cifrado en las comunicaciones internas del sistema y modelos demasiado grandes para ejecutarse en microcontroladores [4, 5]. En consecuencia, el reto no consiste únicamente en maximizar *accuracy*, sino en validar una arquitectura completa que pueda operar sobre hardware restringido, actualizarse de manera distribuida y proteger los artefactos del aprendizaje. Trabajos recientes destacan que los enfoques de TinyML actuales pocas veces incorporan entrenamiento on-board y que, cuando lo hacen, deben combinarlo con FL/Transfer Learning para evitar costosos viajes de datos crudos al servidor [6, 7].

Esta tesis aborda ese reto mediante una arquitectura Edge-Fog-Cloud para detección de intrusiones en tráfico MQTT. La capa Edge se implementa con nodos ESP32-S3 que ejecutan inferencia TinyML y transmiten características de flujo; la capa Fog se implementa sobre Raspberry Pi 4 como gateway MQTT, entrenador local y agregador intermedio; y la capa Cloud se implementa en un PC con FastAPI para coordinar la agregación global. El sistema se evaluó en varias configuraciones: red neuronal MLP con ASCON-128, red neuronal MLP sin ASCON como línea base, variante CNN-1D y estrategia FOG con agregación entre gateways. La elección de ASCON-128 está motivada por su selección reciente como estándar NIST de criptografía ligera para dispositivos restringidos [8, 9], en contraste con esquemas *ad-hoc* basados en permutaciones que han demostrado ser vulnerables a ataques de divulgación de secretos y desincronización [10].

1.2. Problemática

La primera brecha identificada es la **viabilidad de despliegue**. Modelos centralizados como Random Forest o SVM-RBF pueden alcanzar precisiones cercanas al 100 %, pero sus requerimientos de memoria son incompatibles con microcontroladores. La literatura reciente sobre IIoT embebido enfatiza que la integración de modelos AI en plataformas con MCU requiere co-diseño de modelo, herramientas y *stack* de despliegue, y reporta que la mayoría de *frameworks* actuales solo cubren inferencia, no entrenamiento on-board [5, 6]. En contraste, un IDS IoT real necesita inferencia frecuente, bajo consumo de memoria y comunicación limitada [11, 12].

La segunda brecha corresponde a la **privacidad y distribución de datos**. Centralizar capturas PCAP o flujos completos desde múltiples redes introduce costos de transferencia y riesgos de exposición. El aprendizaje federado reduce este problema al compartir pesos o gradientes en lugar de datos crudos, pero en IoT introduce retos adicionales: datos *non-IID*, rondas incompletas, heterogeneidad de hardware y sobrecarga de comunicación [13, 14, 7]. Adicionalmente, los esquemas FL clásicos asumen agregadores capaces (de cientos de MB a GB de RAM), mientras que los gateways IoT con MCU típicos disponen de cientos de KB, lo que impide adoptar agregaciones robustas a clientes maliciosos sin un rediseño jerárquico [15]. La tercera brecha es la **seguridad del propio ciclo federado**. Aunque FL evita mover datos crudos, los pesos del modelo, métricas locales y mensajes de control siguen siendo activos sensibles. Por tanto, las comunicaciones entre ESP32, Raspberry Pi y PC deben proteger confidencialidad e integridad con mecanismos compatibles con dispositivos restringidos. ASCON-128, estandarizado por NIST como criptografía ligera, es una opción pertinente para este escenario [8, 9]. Diseños *ad-hoc* basados en permutaciones simples (XOR, rotaciones) ofrecen menor costo pero han sido comprometidos en análisis recientes, lo cual refuerza la necesidad de adoptar un AEAD estandarizado en el ciclo federado [10].

Finalmente, el sistema debe ser evaluado como arquitectura, no solo como modelo. Por eso se comparan cuatro dimensiones: desempeño del modelo TinyML, costo del aprendizaje federado, overhead de ASCON-128 y efecto de variar el modelo o la topología.

1.3. Justificación

La solución propuesta aporta una validación práctica de un IDS federado para IoT sobre hardware real. A diferencia de un entrenamiento centralizado, el sistema opera con un modelo compacto de 13 características de flujo y tres clases de tráfico: normal, mqtt_bruteforce y scan_A. Este alcance permite instrumentar el ciclo completo de detección, entrenamiento local, agregación global y redistribución del modelo sin depender de infraestructura de alto costo en el borde.

El proyecto también justifica la comparación entre variantes. La rama con ASCON-128 permite medir el costo de proteger todos los canales; la rama sin ASCON sirve como línea

base para aislar el overhead criptográfico; la variante CNN-1D evalúa si una arquitectura más expresiva mejora el desempeño sin romper la viabilidad edge; y el modo FOG introduce una agregación intermedia entre Raspberry Pi antes del servidor central.

El documento está estructurado de la siguiente manera: el Capítulo 1 presenta el problema y los objetivos; el Capítulo 2 resume el marco conceptual y el estado del arte; el Capítulo 3 describe la arquitectura y el protocolo experimental; el Capítulo 4 presenta resultados; el Capítulo 5 concluye y plantea trabajos futuros; y los anexos resumen archivos y artefactos del sistema. Adicionalmente, si desea consultar los scripts de las arquitecturas implementadas en Python y C++ implementadas en este trabajo, puede acceder a ellos a través del siguiente enlace: **Repositorio del proyecto**.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar, implementar y evaluar un prototipo funcional de un sistema de detección de intrusiones en el borde que combine modelos TinyML para inferencia eficiente, aprendizaje federado para entrenamiento colaborativo y cifrado ligero para proteger el intercambio de parámetros y mensajes.

1.4.2. Objetivos Específicos

1. Construir el pipeline de datos para la captura, normalización y generación de features a partir del tráfico y/o de flujos de red, y seleccionar un conjunto de datasets públicos adecuados para entrenamiento y evaluación comparativa.
2. Entrenar y comparar modelos compactos para IDS IoT, incluyendo MLP, CNN-1D, Residual-MLP y Tiny Transformer, exportando los pesos y los parámetros de normalización para ejecución en el microcontrolador ESP32.
3. Implementar un esquema de aprendizaje federado para el entrenamiento distribuido de modelos, evaluando estrategias de agregación, la estabilidad con datos no IID y los costos de comunicación, apoyándose en criterios para la selección de frameworks y extensiones.
4. Integrar mecanismos de cifrado ligero y de autenticación en la arquitectura propuesta y validar experimentalmente el sistema completo, evaluando la capacidad de detección de ataques en los nodos, el impacto del aprendizaje federado frente al entrenamiento centralizado y los efectos en la latencia, el consumo y el desempeño general.

2. Marco Teórico

2.1. Marco Conceptual

2.1.1. IDS en redes IoT/IIoT

Un sistema de detección de intrusiones de red (NIDS) monitorea tráfico y eventos para identificar comportamientos maliciosos o anómalos. En IoT/IIoT, esta tarea es más exigente que en redes tradicionales porque los dispositivos tienen recursos limitados, usan protocolos livianos, presentan patrones de tráfico heterogéneos y suelen estar distribuidos en dominios administrativos distintos. Por ello, la detección debe evaluarse no solo por *accuracy*, sino también por latencia, memoria, comunicación y estabilidad operacional [11, 12]. Surveys recientes sobre seguridad IIoT habilitada por ML insisten en que la diversidad de capas (sensor, gateway, fog, cloud) impide soluciones monolíticas y favorece arquitecturas jerárquicas con responsabilidades repartidas [2, 1]. **Frameworks anteriores combinaron SDN/NFV con ML AGREGAR DEFINICIONES SDN NFV** para coordinar respuesta y monitoreo (*e.g.*, [4, 16]), e incluso frameworks ML basados en cloud para gestionar la seguridad de IoT [17], pero rara vez mostraron despliegue sobre microcontroladores reales.

2.1.2. Representación por flujos

El tráfico de red puede representarse como paquetes, capturas PCAP, logs o flujos. Esta tesis utiliza flujos unidireccionales porque son compactos, tabulares y computables en el borde. Cada flujo se transforma en 13 características numéricas relacionadas con conteo de paquetes, tiempos entre paquetes, tamaños de paquetes, bytes totales y banderas TCP. Esta representación sacrifica parte del detalle secuencial de un PCAP completo, pero permite inferencia TinyML en ESP32-S3 y reduce el volumen de comunicación.

2.1.3. TinyML y modelos compactos

TinyML agrupa técnicas para ejecutar modelos de aprendizaje automático en hardware restringido. En el contexto de esta tesis, el criterio de diseño es que el modelo pueda ejecutarse directamente en microcontroladores ESP32-S3 sin depender de aceleradores externos ni transferir los datos crudos al servidor. La revisión de Dini *et al.* [5] resume el ecosistema actual de *toolchains* (TensorFlow Lite Micro, STM32Cube.AI, Vitis AI, etc.) y muestra

que la integración exitosa depende tanto de la arquitectura del modelo como del soporte HW/SW del MCU. Ficco *et al.* [6] demostraron que el entrenamiento on-board en dispositivos como ESP32 y Arduino MKR1010 es viable cuando se combina FL con *transfer learning*, alcanzando accuracy comparable al entrenamiento centralizado (86.48 % en clasificación) con latencia y consumo controlados. Los modelos evaluados en esta tesis incluyen MLP, CNN-1D, Residual-MLP y Tiny Transformer; sin embargo, el despliegue principal utiliza un MLP compacto y una CNN-1D debido a su equilibrio entre tamaño, precisión y simplicidad de inferencia, en línea con las recomendaciones de la literatura para clasificación tabular en MCUs [18, 5].

2.1.4. Aprendizaje federado jerárquico

El aprendizaje federado (FL) permite entrenar modelos de forma distribuida compartiendo actualizaciones en lugar de datos crudos. En escenarios IoT, una arquitectura jerárquica (HFL) resulta natural porque los nodos edge pueden comunicarse con gateways cercanos y estos, a su vez, con un coordinador central. Este diseño reduce exposición de datos, organiza la comunicación por capas y permite agregar actualizaciones localmente antes de enviarlas al servidor global [19, 13]. Hajj *et al.* [20] muestran que un IDS federado *cross-layer* para IoT puede alcanzar precisión competitiva con menor consumo de comunicación, y Chaurasia *et al.* [21] evidencian beneficios al usar redes residuales como base del cliente FL en IIoT.

La literatura advierte que FL en IoT enfrenta datos *non-IID*, conectividad irregular y heterogeneidad de hardware [14, 7]. En esta tesis, esas condiciones se reproducen parcialmente mediante nodos con distribuciones distintas: un nodo genera tráfico normal y otro genera una mezcla de tráfico normal, fuerza bruta MQTT y escaneo. Adicionalmente, Tran *et al.* [15] cuantifican una brecha de memoria de hasta tres órdenes de magnitud entre los métodos de agregación bizantino-resilientes (512 MB–2 GB) y los MCU comerciales (~264 KB de SRAM), evidenciando la necesidad de descomponer el problema en capas. **La arquitectura de esta tesis adopta esa filosofía sin requerir blockchain ni consenso PoP, dejando la robustez bizantina como trabajo futuro.**

2.1.5. Cifrado ligero y ASCON-128

Aunque FL reduce la centralización de datos, los pesos del modelo y mensajes de control siguen siendo sensibles. Un atacante que intercepte o modifique actualizaciones podría inferir información, degradar el modelo o introducir comportamiento malicioso. Por esta razón, la arquitectura integra ASCON-128 como cifrado autenticado ligero para proteger confidencialidad e integridad en los canales MQTT y HTTP del ciclo federado.

ASCON fue seleccionado por NIST en febrero de 2023 como estándar de criptografía ligera, después de un proceso multi-anual frente a candidatos como GIFT-COFB, Sparkle, Romulus

y TinyJambu [8, 9]. Internamente, ASCON aplica una permutación de 320 bits estructurada en cinco palabras de 64 bits, con dos variantes principales: ASCON-128 (rate 64 bits) y ASCON-128a (rate 128 bits). El survey de Kaur *et al.* [9] resume implementaciones FPGA/ASIC, ataques diferenciales, ataques de canal lateral y contramedidas, ofreciendo una guía consolidada para diseñadores de sistemas embebidos. Su diseño resulta apropiado para mensajes pequeños como features, pesos parciales y modelos globales [8, 22].

Una alternativa frecuente en la industria son los protocolos *ultra-lightweight* basados en permutaciones *ad-hoc* (XOR, rotaciones, shifts), pero Mirzaali *et al.* [10] demuestran experimentalmente sobre tres Raspberry Pi que estos esquemas suelen ser vulnerables a ataques de divulgación completa de secretos y desincronización, incluso cuando la estructura del protocolo parece sólida. En esta tesis se mide explícitamente su overhead frente a una versión funcionalmente equivalente sin cifrado, replicando el patrón experimental de varios trabajos previos sobre Edge-Cloud con cifrado por atributos [23].

2.1.6. Agregación FOG

El modo FOG extiende la arquitectura estándar con comunicación entre gateways Raspberry Pi. Un gateway líder recibe actualizaciones de gateways pares mediante MQTT, realiza un FedAvg intermedio y envía al servidor central una actualización preagregada. Este esquema aproxima escenarios reales con múltiples zonas o sedes, donde una capa fog reduce tráfico hacia la nube y permite coordinación local antes de la agregación global. Wang *et al.* [24] discuten cómo la fusión de datos en sistemas ciber-físico-sociales se beneficia de pre-agregación cercana a la fuente, y EdgeTrust-Shard [15] demuestra que distribuir complejidad computacional por la topología es indispensable cuando los participantes son MCUs. El presente trabajo opera en una jerarquía de tres niveles (PC, Pi, ESP32) que materializa esa idea sin introducir blockchain.

2.2. Estado del Arte

La literatura revisada se agrupa en cuatro líneas, articuladas alrededor de las cuatro dimensiones del sistema (Edge, Fog/FL, criptografía y datasets/etiquetado).

2.2.1. IDS y TinyML para IoT/IIoT

La primera línea aborda IDS para IoT/IIoT y destaca que los modelos de alta precisión no siempre son desplegables en el borde. Trabajos recientes sobre TinyML e IDS enfatizan que la memoria, la latencia y el costo energético deben reportarse junto con las métricas de clasificación [11, 25, 12, 18]. Surveys más generales sobre IIoT, DL y big data en seguridad IoT [1, 2] muestran que la mayoría de los pipelines reportados se apoyan en servidores capaces y carecen de despliegue real en MCUs. La revisión de Dini *et al.* [5] compila explícitamente

la oferta de *toolchains* TinyML (TFLM, STM32Cube.AI, Vitis AI) y los compromisos al integrarlos en plataformas embebidas.

2.2.2. Aprendizaje federado para seguridad IoT

La segunda línea corresponde al aprendizaje federado aplicado a seguridad IoT. FL permite entrenar modelos sin centralizar datos, pero introduce retos de comunicación, datos *non-IID* y participación parcial. Estudios recientes sobre FL para IDS muestran que la precisión puede degradarse frente al entrenamiento centralizado, pero que la reducción de datos compartidos puede justificar el intercambio en escenarios con restricciones de privacidad [13, 14, 20, 26]. Chaurasia *et al.* [21] usan redes residuales como cliente FL en IIoT, mientras que Ficco *et al.* [6] validan FL+TL para entrenamiento on-board en MCUs y Raspberry Pi 3, comparando contra TFLite. EdgeTrust-Shard [15] aporta una visión más reciente: cuando los participantes son MCUs, las defensas Byzantine-robust convencionales (Krum, FLTrust, Fed-NGA) son inalcanzables sin descomponer el problema en una jerarquía como la propuesta en este trabajo.

2.2.3. Criptografía ligera y autenticación

La tercera línea es criptografía ligera. ASCON y otros esquemas de bajo costo buscan proteger dispositivos restringidos sin introducir overhead prohibitivo. Para un IDS federado, esta línea es especialmente relevante porque las actualizaciones de modelo son parte del activo de seguridad y deben protegerse durante el tránsito [8, 22, 9]. El survey de Kaur *et al.* [9] provee una taxonomía de implementaciones FPGA/ASIC, ataques (diferenciales, cube, fault injection, side-channel) y contramedidas, lo que orienta decisiones de diseño y trabajos futuros. En contraste, Mirzaali *et al.* [10] muestran cómo permutaciones ultra-ligeras *ad-hoc* para autenticación RFID/IoT se rompen experimentalmente en hardware (RPi + Wi-Fi). Mecanismos complementarios de control de acceso [27] y cifrado por atributos [23] se han propuesto para escenarios sanitarios y de smart cities, pero ninguno integra cifrado y FL sobre la misma plataforma.

2.2.4. Datasets y representación del tráfico

La cuarta línea cubre datasets y representación de tráfico. UNSW-NB15, Bot-IoT, TON_IoT, Edge-IIoTset, IoT-23 y CICIoT2023 [28, 29, 30, 31, 32, 33, 34] ofrecen capturas etiquetadas con diversos protocolos y vectores de ataque. Trabajos recientes [35, 36] proponen consolidar/integrar varios datasets para evitar sobreajuste a una sola fuente. En esta tesis se trabaja con tres datasets de flujos unidireccionales (`normal`, `mqtt_bruteforce`, `scan_A`) representativos del tráfico que un nodo MQTT puede observar en un laboratorio o smart home.

2.3. Síntesis de la Brecha

Los trabajos existentes suelen cubrir por separado TinyML, FL o cifrado ligero. La brecha abordada en esta tesis es la integración y medición de un ciclo completo Edge–Fog–Cloud sobre hardware real, con inferencia en ESP32-S3, entrenamiento local en Raspberry Pi, agregación global en PC, protección ASCON-128, línea base sin cifrado, variante CNN y modo FOG. Esta evaluación conjunta permite discutir el sistema como una arquitectura operativa y no solo como un clasificador aislado. Cuatro contribuciones específicas distinguen este trabajo:

CAMBIAR ESTO POR 2 PARRAFICOS QUE EXPLIQUES ESTAS 4 IDEAS, QUE DESARROLLE MAS LA IDEA Y NO SE CENTRE EN UN PASO A PASO

1. Implementación interoperable de ASCON-128 en C++ (ESP32) y Python (RPi/PC), instrumentada con métricas de tiempo y tamaño por canal y por ronda.
2. Comparación experimental directa, sobre el mismo hardware y mismas condiciones, entre la rama con ASCON y la rama sin ASCON.
3. Comparación de dos familias de modelos federables (MLP/RN y CNN-1D) bajo el mismo protocolo HFL, con un Tiny Transformer y un Residual MLP como referencias offline.
4. Estrategia FOG con agregación intermedia entre gateways Raspberry Pi (líder/peer), que no aparece reportada de forma combinada con ASCON-128 en la literatura revisada.

Table 2-1.. Relación entre literatura y decisiones de diseño.

Línea	Referencias	Uso en la tesis
TinyML / IDS IoT	[11, 12, 25, 18, 5]	Selección de modelos compactos, reporte de huella de memoria, latencia de inferencia en ESP32-S3 y elección de <i>toolchain</i> de despliegue.
Surveys IoT/IIoT seguridad	[1, 2, 4, 16]	Encuadre del problema, justificación de jerarquía Edge–Fog–Cloud y comparación contra enfoques no edge.
FL / HFL para IDS	[19, 13, 14, 7, 20, 26, 21, 6]	Diseño de FedAvg jerárquico, discusión de datos <i>non-IID</i> y sustento del entrenamiento on-board en MCUs/RPi.

Línea	Referencias	Uso en la tesis
Robustez/escalabilidad[15] FL en MCUs		Justificación de la jerarquía y delimitación de las amenazas no cubiertas (clientes bizantinos).
Criptografía ligera	[8, 22, 9, 10, 37]	Integración de ASCON-128, comparación contra baseline sin cifrado, justificación de no usar permutaciones <i>ad-hoc</i> y delimitación de canales laterales como trabajo futuro.
Datasets IoT-IDS	[28, 34, 32, 33, 35, 36]	Selección de las 13 features de flujo y de las tres clases (<code>normal</code> , <code>mqtt_bruteforce</code> , <code>scan_A</code>).
Pre-agregación / data fusion	[24]	Justificación conceptual del modo FOG (FedAvg de gateways antes del PC).

3. Metodología

La presente investigación sigue un enfoque experimental y cuantitativo, estructurado en seis fases: (i) construcción del pipeline de datos, (ii) entrenamiento y comparación offline de modelos livianos, (iii) implementación de la arquitectura jerárquica de aprendizaje federado, (iv) integración de cifrado ligero ASCON-128, (v) construcción de una línea base sin cifrado y (vi) evaluación de variantes con MLP/RN, CNN-1D y estrategia FOG sobre hardware real.

3.1. Modelo de Amenazas y Suposiciones

Antes de detallar la arquitectura, se hace explícito el modelo de amenazas que motiva las decisiones de diseño. Se asume un atacante de red pasivo o activo capaz de observar y modificar el tráfico inalámbrico entre nodos ESP32, gateways Raspberry Pi y el PC coordinador, pero *externo* al sistema. Las amenazas que el sistema busca mitigar son:

- **Confidencialidad de pesos y features:** Un atacante que capture los mensajes MQTT/HTTP no debe poder leer el contenido (pesos federados, métricas locales, vectores de features). Esto se garantiza con ASCON-128 sobre todos los canales [8, 9].
- **Integridad y autenticidad:** Mensajes alterados o inyectados deben ser detectados y descartados. Se usa el tag de autenticación de 128 bits que ASCON-128 produce de forma nativa, evitando el patrón frecuente de combinar cifrado simétrico con HMAC externo [37].
- **Reuso de mensajes (replay):** Cada nonce se construye a partir de un timestamp truncado a 32 bits y un contador monótono local; combinado con el tag autenticado, esto detecta reutilizaciones aunque un atacante reenvíe un mensaje válido.
- **Diseños criptográficos no estandarizados:** Se descarta el uso de permutaciones ultra-ligeras *ad-hoc* para autenticación, dado que análisis recientes han demostrado vulnerabilidades estructurales explotables sobre Raspberry Pi en condiciones realistas [10].

Amenazas fuera del alcance. Se delimitan explícitamente las siguientes amenazas que *no* se mitigan en esta versión del sistema:

- **Clientes bizantinos / data poisoning:** Un gateway o nodo edge comprometido podría enviar pesos manipulados que sesguen el modelo global. La literatura reporta que un 10 % de participantes maliciosos puede degradar la accuracy global de 96 % a 23 % [15]. Mitigaciones (Krum, FLTrust, Fed-NGA, MF-PoP) requieren capacidades de

cómputo y memoria que exceden las de un MCU sin descomposición arquitectónica adicional. Esta línea queda como trabajo futuro.

- **Ataques de canal lateral sobre ASCON:** Análisis de potencia, EM o fault injection contra la implementación ASCON pueden recuperar la clave si el atacante tiene acceso físico al MCU [9]. El sistema asume despliegue en un entorno físico controlado.
- **Inferencia de membresía:** Aunque FL no comparte datos crudos, los pesos pueden filtrar información sobre los datos locales. Defensas como differential privacy (DP-SGD) no se aplican en esta versión.
- **Compromiso del PC coordinador:** El PC se asume confiable (TCB). Trabajos basados en blockchain o multi-coordinador podrían eliminar este supuesto [15].

3.2. Arquitectura del Sistema

El sistema propuesto se organiza en tres capas jerárquicas que reflejan la topología Edge-Fog-Cloud, cada una con roles diferenciados de procesamiento, entrenamiento y coordinación.

3.2.1. Capa Edge: Nodos ESP32-S3

Los nodos de borde se implementan sobre microcontroladores **ESP32-S3** con conectividad Wi-Fi nativa. Cada nodo cumple tres funciones:

1. **Generación y captura de tráfico:** simula flujos de red MQTT con patrones estadísticos calibrados a partir de los datasets reales (Sección 3.3).
2. **Extracción de características:** computa en tiempo real un vector de 13 features por flujo (Tabla 3-2).
3. **Inferencia local (TinyML):** ejecuta un modelo compacto directamente en el microcontrolador, clasificando cada flujo en tres categorías: `normal`, `mqtt_bruteforce` y `scan_A`.

Se despliegan dos nodos diferenciados: un nodo *normal* que genera exclusivamente tráfico benigno, y un nodo *simulado* que genera una mezcla de 40 % normal, 30 % bruteforce y 30 % scan_A, reproduciendo las distribuciones estadísticas reales de los datasets. En la rama RN se usa un MLP; en la rama CNN se usa una CNN-1D con capas convolucionales fijas y capas densas federadas.

3.2.2. Capa Fog: Gateway Raspberry Pi 4

La Raspberry Pi 4 actúa como *gateway* Fog y ejecuta `gateway_hfl.py` o `gateway_hfl_fog.py`, según el escenario experimental. Sus responsabilidades incluyen:

- Actuar como **broker MQTT** (Mosquitto) para recibir las features de los ESP32.
- Mantener un **buffer de entrenamiento** de $N_s = 30$ muestras etiquetadas mediante una función heurística basada en las distribuciones reales del dataset.

- **Entrenar localmente** un modelo Keras/TensorFlow idéntico al del ESP32 (5 épocas, Adam con $\eta = 0,005$, batch=8).
- **Transmitir los pesos** de las capas federadas al servidor central vía HTTP.
- **Distribuir el modelo global** recibido del servidor a los ESP32 vía MQTT.
- En modo FOG, **agregar pesos entre gateways Raspberry Pi** antes de enviar una actualización preagregada al PC.

3.2.3. Capa Cloud: Servidor PC

Un PC convencional ejecuta `server_hfl.py` o `server_hfl_fog.py` (FastAPI + Uvicorn) y coordina el aprendizaje federado global:

- Recibe los pesos locales de K gateways (en esta implementación, $K = 2$).
- Aplica el algoritmo **FedAvg** (Sección 3.6) ponderado por número de muestras.
- Distribuye el modelo global actualizado a todos los gateways.
- Proporciona un **dashboard analítico** en tiempo real con gráficas de accuracy y loss por ronda.

3.2.4. Modos de operación

La operación de la arquitectura fue implementada en dos modos:

- **Modo estándar:** cada Raspberry Pi actúa como gateway independiente. El servidor PC espera actualizaciones de $K = 2$ gateways y aplica FedAvg global.
- **Modo FOG:** una Raspberry Pi líder recibe pesos de una Raspberry Pi par mediante MQTT, calcula FedAvg local de la capa fog y envía al PC una actualización preagregada. El modelo global retorna al líder y se redistribuye a los peers y a los ESP32.

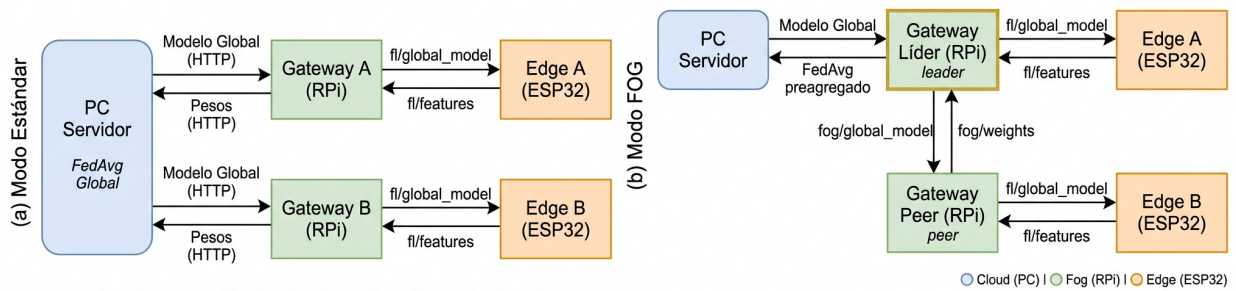


Figure 3-1.. Modos de operación: (a) estándar con agregación directa PC–gateways y (b) FOG con agregación jerárquica mediante un gateway líder.

3.3. Conjunto de Datos

Se utilizan tres datasets públicos de flujos de red IoT, cada uno representando una clase de tráfico. Estos provienen de capturas PCAP procesadas y convertidas a representaciones de

flujo unidireccional (*uniflow*):

Table 3-1.. Composición del dataset de flujos de red.

Dataset	Flujos	Etiqueta	Protocolo principal
uniflow_normal.csv	171 837	0 (normal)	MQTT, TCP
uniflow_mqtt_bruteforce.csv	33 080	1 (mqtt_bruteforce)	MQTT (login)
uniflow_scan_A.csv	51 359	2 (scan_A)	TCP SYN
Total	256 276	3 clases	—

Cada flujo se representa mediante un vector de $d = 13$ características numéricas, seleccionadas para ser computables en tiempo real por un microcontrolador:

Table 3-2.. Vector de 13 features extraídas por flujo.

Índice	Feature	Descripción
0	num_pkts	Número de paquetes del flujo
1	mean_iat	Inter-arrival time promedio (s)
2	std_iat	Desviación estándar del IAT
3	min_iat	IAT mínimo
4	max_iat	IAT máximo
5	mean_pkt_len	Longitud promedio de paquete (bytes)
6	num_bytes	Total de bytes del flujo
7	num_psh_flags	Conteo de flags PSH
8	num_rst_flags	Conteo de flags RST
9	num_urg_flags	Conteo de flags URG
10	std_pkt_len	Desviación estándar de longitud de paquete
11	min_pkt_len	Longitud mínima de paquete
12	max_pkt_len	Longitud máxima de paquete

Para el preprocesamiento, se aplica un **StandardScaler** que normaliza cada feature a media cero y varianza unitaria. Los parámetros del scaler (μ_i, σ_i) se exportan como constantes embebidas en el firmware del ESP32, permitiendo la normalización en el dispositivo sin dependencia de bibliotecas externas.

3.4. Modelo de Aprendizaje: MLP Multiclase

Se diseñó un perceptrón multicapa (MLP) compacto orientado a la ejecución en microcontroladores con restricciones de memoria:

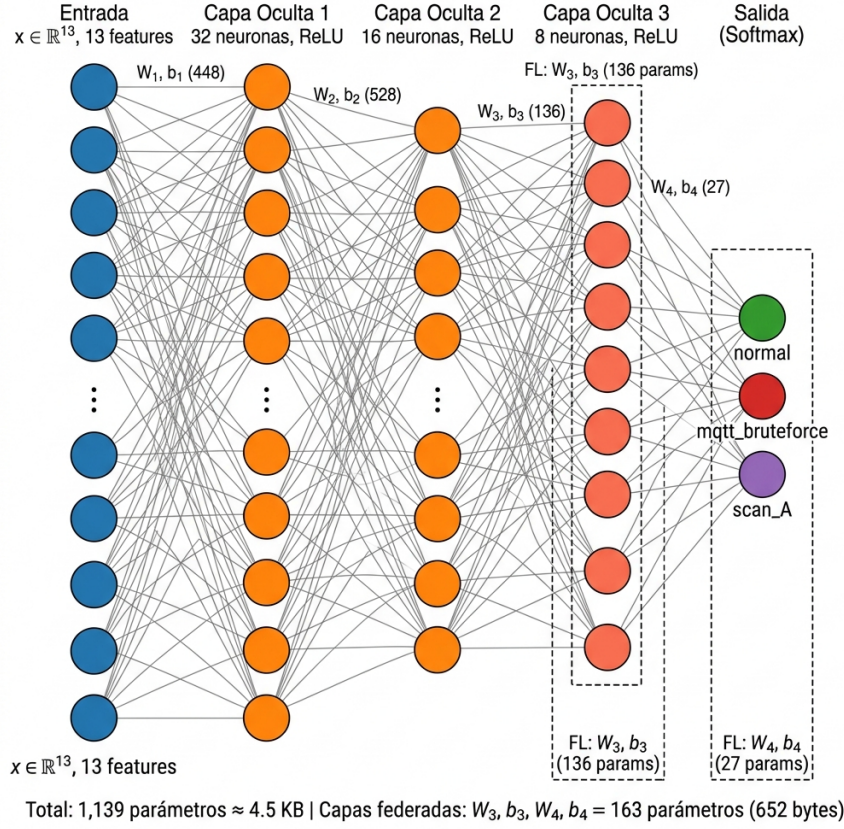


Figure 3-2.. Arquitectura del modelo MLP multiclase. Las capas ocultas 3 y de salida (recuadro punteado) conforman las *capas federadas* (W_3, b_3, W_4, b_4), cuyos 163 parámetros son los únicos intercambiados durante el aprendizaje federado.

Las primeras tres capas utilizan activación ReLU y la capa de salida emplea Softmax para obtener probabilidades sobre las 3 clases. La Tabla **3-3** detalla la distribución de parámetros:

Table 3-3.. Distribución de parámetros del modelo MLP.

Capa	Entrada	Salida	Parámetros
Dense 1 (ReLU)	13	32	$13 \times 32 + 32 = 448$
Dense 2 (ReLU)	32	16	$32 \times 16 + 16 = 528$
Dense 3 (ReLU)	16	8	$16 \times 8 + 8 = 136$
Dense 4 (Softmax)	8	3	$8 \times 3 + 3 = 27$
Total			1 139

Con 1 139 parámetros en punto flotante de 32 bits, el modelo completo ocupa $\approx 4,5$ KB de memoria, lo que permite su ejecución directa en la SRAM del ESP32-S3 sin necesidad de cuantización.

En el esquema de aprendizaje federado, solo las **dos últimas capas** (W_3, b_3, W_4, b_4 , 163 parámetros) son intercambiadas entre nodos, reduciendo el costo de comunicación a ≈ 652 bytes por actualización (sin serialización JSON).

3.5. Modelos livianos evaluados

Antes del despliegue federado se ejecutó un notebook general de entrenamiento con cuatro familias de modelos sobre las mismas 13 características y las mismas tres clases. El objetivo fue seleccionar una arquitectura viable para ESP32-S3 y, al mismo tiempo, documentar alternativas para comparación experimental:

Table 3-4.. Modelos livianos entrenados y artefactos exportados.

Modelo	Arquitectura	Artefactos exportados
MLP/RN	$13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$	<code>model_weights.h</code> , <code>ids_3class.keras</code> , scaler
CNN-1D	$\text{Conv1D}(32) \rightarrow \text{Conv1D}(16) \rightarrow \text{GAP} \rightarrow \text{Dense8} \rightarrow \text{Dense3}$	Pesos Conv fusionados + capas densas federadas
Residual-MLP	MLP compacto con conexión residual	Pesos, scaler y modelo Keras
Tiny Transformer	Auto-atención tabular liviana	Pesos y modelo Keras para análisis offline

El despliegue principal selecciona el MLP/RN porque ofrece el mejor balance entre precisión offline, tamaño y simplicidad de implementación en C++ sobre ESP32-S3. La variante CNN-1D se implementa como rama experimental: las capas convolucionales permanecen fijas en el edge y solo se federan las capas densas `dense_1` y `dense_out`, equivalentes funcionalmente a las capas federadas del MLP. Esta estrategia de *federar solo las capas finales* se reporta también en trabajos previos de FL+TL para TinyML [6, 26] y permite acotar el tamaño del payload sin perder capacidad de adaptación local.

3.6. Protocolo de Aprendizaje Federado Jerárquico

Se implementa un esquema de aprendizaje federado jerárquico (HFL) con flujo bidireccional:

1. **Nivel Edge (ESP32 $\rightarrow$ RPi):** Los nodos ESP32 envían vectores de features al gateway cada 5 segundos vía MQTT (`f1/features`). El gateway acumula $N_s = 40$ muestras etiquetadas heurísticamente.

2. **Nivel Fog (RPI $\rightarrow$ PC o RPI $\rightarrow$ RPI líder):** Tras llenar el buffer, el gateway entrena localmente durante $E = 5$ épocas y envía las capas federadas junto con la accuracy local y el número de muestras.

3. **Nivel Cloud (Agregación FedAvg):** El servidor espera recibir actualizaciones de $K = 2$ gateways y aplica FedAvg ponderado por número de muestras:

$$W_{\text{global}}^{(t+1)} = \frac{\sum_{k=1}^K n_k \cdot W_k^{(t)}}{\sum_{k=1}^K n_k} \quad (3-1)$$

donde n_k es el número de muestras usadas para el entrenamiento local del gateway k y $W_k^{(t)}$ sus pesos locales en la ronda t .

El modelo global resultante es distribuido de vuelta a los gateways (HTTP POST a `/deploy-model`), quienes a su vez lo publican a los ESP32 por MQTT (`f1/global_model`). En la rama sin ASCON se usan topics y endpoints con sufijo `_plain`, por ejemplo `f1/features_plain`, `f1/global_model_plain`, `/aggregate-from-gateway-plain` y `/deploy-model-plain`. La Figura **3-3** resume el flujo bidireccional completo en sus tres fases canónicas (top-down deployment, recolección y entrenamiento local, y agregación bottom-up).

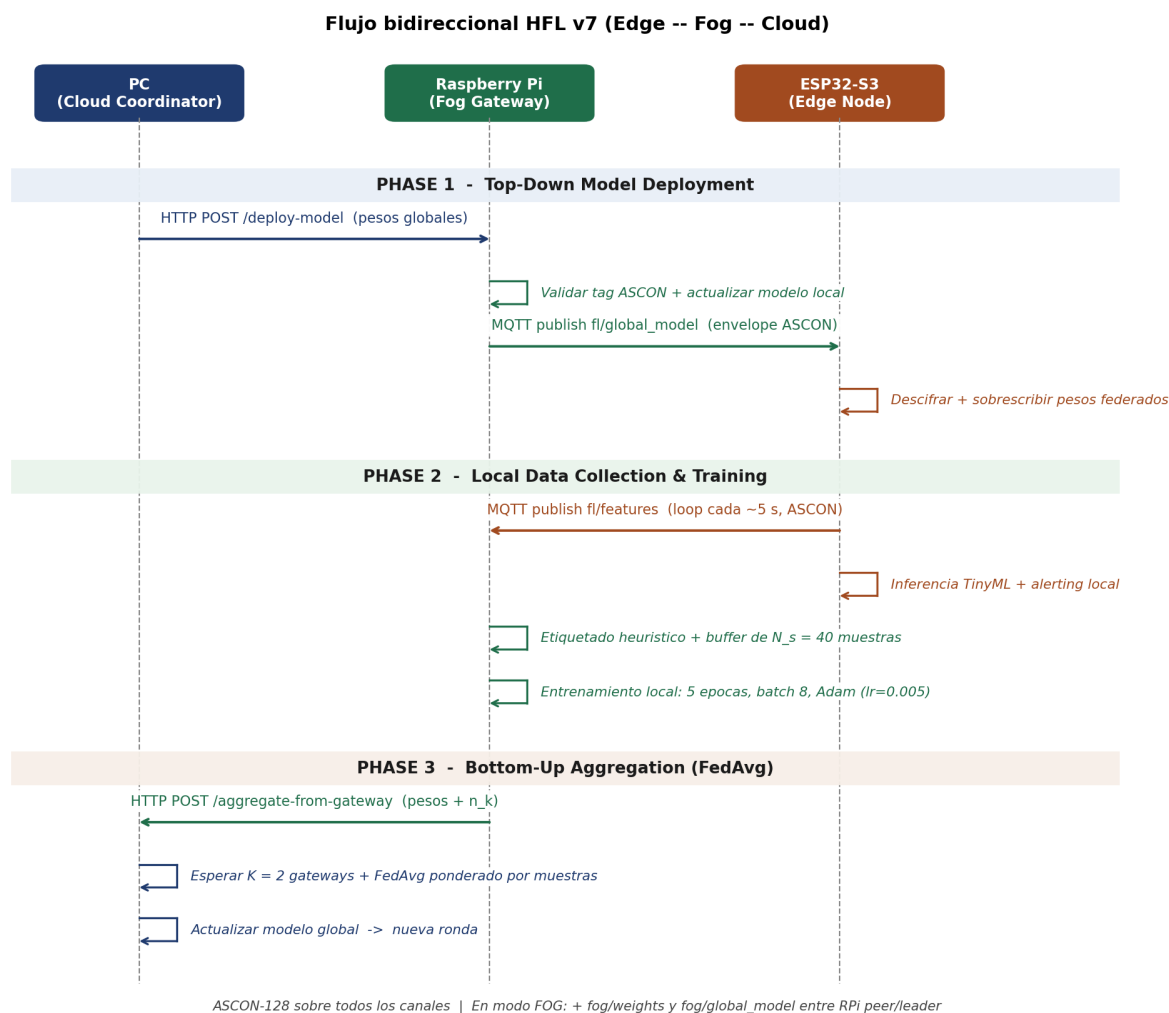


Figure 3-3.. Diagrama de secuencia del protocolo HFL v7 bidireccional. Los actores PC (Cloud), Raspberry Pi (Fog) y ESP32-S3 (Edge) intercambian mensajes en tres fases: despliegue del modelo global, recolección de features con entrenamiento local y agregación FedAvg. Todos los canales MQTT/HTTP se protegen con ASCON-128 cuando la variante está habilitada; en modo FOG se añaden los topics `fog/weights` y `fog/global_model` entre Raspberry Pi peer y leader.

En modo FOG, el flujo agrega dos canales adicionales: `fog/weights` para que los peers envíen pesos al líder y `fog/global_model` para que el líder redistribuya el modelo global a los peers. El etiquetado en el gateway se realiza mediante una **función heurística** calibrada con las distribuciones estadísticas reales de los datasets:

- Si $\text{num_pkts} \geq 50$ y $\text{num_psh} \geq 10$: etiqueta = 1 (bruteforce).
- Si $\text{num_pkts} \leq 5$ y $\text{mean_pkt_len} \leq 50$ y $\text{num_psh} \leq 1$: etiqueta = 2 (scan A).
- Si $\text{num_pkts} \leq 30$ y $\text{mean_pkt_len} \geq 50$: etiqueta = 0 (normal).

- En otro caso, la muestra se descarta (-1) .

3.7. Integración de Cifrado Ligero ASCON-128

Para proteger los artefactos de aprendizaje federado en tránsito, se integra **ASCON-128** [8], el estándar de criptografía ligera seleccionado por NIST para dispositivos restringidos. ASCON provee cifrado autenticado con datos asociados (AEAD), garantizando simultáneamente *confidencialidad* e *integridad* de los mensajes.

3.7.1. Parámetros Criptográficos

- **Clave simétrica:** 128 bits (16 bytes), pre-compartida entre todos los dispositivos.
- **Nonce:** 128 bits, generado a partir de timestamp truncado a 32 bits y un contador monotónico.
- **Tag de autenticación:** 128 bits (16 bytes).
- **Tasa (rate):** 64 bits (8 bytes por bloque).
- **Permutación:** p^{12} para inicialización/finalización, p^6 para procesamiento de bloques.

Se elige ASCON-128 (rate de 64 bits) sobre la variante ASCON-128a (rate de 128 bits) por dos razones: (i) su mayor número de rondas en la fase de procesamiento del cuerpo proporciona un margen de seguridad superior frente a ataques diferenciales y de cubo recientemente revisados [9]; y (ii) los payloads del sistema (features de 13 floats, vectores de pesos de ~ 3.6 KB) son lo suficientemente pequeños para que la diferencia de *throughput* entre ASCON-128 y ASCON-128a no sea determinante. Esta decisión es alineada con la recomendación de NIST SP 800-232 para canales con paquetes pequeños [8].

3.7.2. Canales Cifrados

Se cifran los **cuatro canales de comunicación** del sistema:

Table 3-5.. Canales de comunicación protegidos con ASCON-128.

Canal	Protocolo	Contenido	Implementación
ESP32 $\rightarrow$ RPi	MQTT	Features (13 floats)	C++ (<code>ascon128.h</code>)
RPi $\rightarrow$ PC	HTTP	Pesos locales + métricas	Python (<code>ascon128.py</code>)
PC $\rightarrow$ RPi	HTTP	Modelo global	Python (<code>ascon128.py</code>)
RPi $\rightarrow$ ESP32	MQTT	Modelo global	Python $\rightarrow$ C++

El payload cifrado se encapsula en un *envelope* JSON con tres campos codificados en Base64:

```
{"ct": "<ciphertext_b64>", "tag": "<tag_b64>", "nonce": "<nonce_b64>"}
```

3.7.3. Métricas de Overhead

Para cuantificar el overhead de ASCON, se instrumentan todas las operaciones de cifrado y descifrado con temporizadores de alta resolución (`micros()` en ESP32, `time.perf_counter()` en Python) y se registran automáticamente en archivos CSV mediante el módulo `ascon_metrics.py`, capturando:

- Tiempo de cifrado/descifrado (ms).
- Tamaño del plaintext y del envelope cifrado (bytes).
- Overhead de comunicación (bytes adicionales y porcentaje).
- Ronda de FL asociada.

Adicionalmente, se mantiene una **rama sin cifrado** (`hfl_v7-no-ascon`) del repositorio con idéntica funcionalidad pero sin ASCON, permitiendo la comparación directa de tiempos de comunicación como línea base. En esta rama los mensajes se serializan como JSON plano y se registran mediante `plain_metrics.py`.

3.8. Variantes implementadas en hfl_v7

Table 3-6.. Variantes de arquitectura evaluadas.

Carpeta	Modelo	Seguridad	Topología
<code>hfl_v7-RN</code>	MLP/RN	ASCON-128	Estándar y FOG
<code>hfl_v7-no-ascon</code>	MLP/RN	JSON plano	Estándar y FOG
<code>hfl_v7-CNN</code>	CNN-1D	ASCON-128	Estándar y FOG

Estas variantes permiten aislar tres efectos: el impacto del cifrado, el impacto de cambiar el modelo y el impacto de introducir una agregación fog intermedia.

3.9. Modelos Clásicos de ML como Línea Base

Para responder a la pregunta de si TinyML puede alcanzar precisión comparable a modelos tradicionales, se entrenan y optimizan cuatro algoritmos de ML supervisado sobre los mismos datasets:

1. **Decision Tree:** Optimizado con `GridSearchCV` sobre `max_depth`, `min_samples_split`, `min_samples_leaf` y `criterion`. Incluye `class_weight='balanced'` para manejar el desbalance de clases.
2. **Random Forest:** Optimizado con `RandomizedSearchCV` ($n = 80$ iteraciones) sobre `n_estimators`, `max_depth`, `max_features` y variantes de `class_weight`.
3. **Logistic Regression:** Evaluada con penalizaciones L1, L2 y ElasticNet, variando el parámetro de regularización C .
4. **Support Vector Machine (SVM):** Se comparan kernel lineal y RBF. Para el kernel RBF, se aplica submuestreo estratificado (máximo 25 000 muestras por clase) para evitar tiempos de entrenamiento prohibitivos.

Todos los modelos se evalúan con validación cruzada estratificada ($k = 5$) usando F1-Score macro como métrica de selección. Para estimar la viabilidad de despliegue en ESP32, se utiliza la biblioteca `m2cgen` para exportar los modelos entrenados a código C puro, y se mide:

- Tamaño estimado en memoria del modelo exportado.
- Tiempo de inferencia promedio (100 iteraciones con warmup).
- Métricas: Accuracy, F1 macro, F1 weighted, Precision, Recall y MCC (Matthews Correlation Coefficient).

3.10. Protocolo Experimental

La evaluación se organiza en cinco escenarios experimentales:

3.10.1. Escenario 1: Sistema HFL Completo con ASCON

Se despliega el sistema completo en hardware real:

- $2 \times$ ESP32-S3 (nodo normal + nodo simulado).
- $1 \times$ Raspberry Pi 4 (gateway, broker MQTT, entrenamiento local).
- $1 \times$ PC (servidor federado, dashboard).
- Red Wi-Fi local compartida.
- ASCON-128 habilitado en todos los canales.

Se ejecutan ≥ 50 rondas de aprendizaje federado, registrando accuracy global, loss, tiempos de cifrado/descifrado y tamaños de mensajes.

3.10.2. Escenario 2: Sistema HFL sin ASCON (Baseline)

Idéntico al Escenario 1 pero con ASCON deshabilitado, utilizando la rama `hfl_v7-no-ascon`. La comunicación se realiza en JSON plano. Se miden los mismos indicadores para calcular el overhead atribuible exclusivamente al cifrado.

3.10.3. Escenario 3: Sistema HFL con CNN-1D

Se ejecuta la misma arquitectura HFL, pero sustituyendo el MLP por una CNN-1D. En este caso, las capas Conv1D y BatchNorm fusionadas permanecen fijas en el ESP32, mientras que las capas densas finales se actualizan mediante FL.

3.10.4. Escenario 4: Estrategia FOG

Se despliega `gateway_hfl_fog.py` con roles `leader` y `peer`. El líder agrega pesos locales y pesos de peers mediante FedAvg fog, envía una actualización preagregada al PC y redistribuye el modelo global hacia peers y ESP32.

3.10.5. Escenario 5: Modelos Clásicos Centralizados

Los modelos clásicos (Sección 3.9) se entrenan de forma centralizada en el PC usando la totalidad del dataset, con partición 80/20 estratificada. Esto establece un techo de rendimiento (*upper bound*) contra el cual comparar la precisión del sistema federado.

3.10.6. Métricas de Evaluación

- **Precisión de detección:** Accuracy, F1-Score (macro y weighted), Precision, Recall, MCC.
- **Overhead criptográfico:** Tiempo de cifrado/descifrado (ms), incremento de tamaño (bytes, %), overhead como porcentaje del ciclo total de FL.
- **Eficiencia federada:** Número de rondas hasta convergencia, accuracy por ronda, costo de comunicación por ronda.
- **Viabilidad edge:** Huella de memoria del modelo (KB), tiempo de inferencia en ESP32 (μ s), consumo de SRAM.
- **Efecto de variantes:** comparación entre MLP/RN, CNN-1D, baseline sin ASCON y topología FOG.

4. Resultados

En este capítulo se presentan los resultados experimentales del sistema de detección de intrusiones desplegado en hardware real, organizados en torno a las preguntas de investigación formuladas en el Capítulo 1. Primero se analiza el desempeño de los modelos clásicos como línea base centralizada. Luego se comparan los modelos livianos entrenados en el notebook general. Posteriormente se evalúan las **ocho variantes HFL** `hfl_v7-RN`, `hfl_v7-no-ascon-RN`, `hfl_v7-CNN` y `hfl_v7-no-ascon-CNN`, cada una con sus modos `Results/` (NoFOG) y `Results_FOG/`, integradas en una capa SRE unificada (`Analisis de Modelos/Completo/`) que cubre 58 intentos. Finalmente, se cuantifica el overhead del cifrado ASCON-128 contrastando las cuatro variantes con cifrado contra las cuatro PLAIN.

4.1. Desempeño de Modelos Clásicos (Línea Base Centralizada)

Los cuatro modelos clásicos de ML fueron entrenados sobre la totalidad del dataset (256 276 flujos) con partición estratificada 80/20 y optimización de hiperparámetros mediante validación cruzada ($k = 5$). La Tabla 4-1 resume los resultados obtenidos.

Table 4-1.. Desempeño de modelos clásicos de ML (entrenamiento centralizado).

Modelo	Accuracy	F1 Macro	F1 Weighted	Precision	Recall	MCC
Decision Tree	0.9987	0.9981	0.9987	0.9982	0.9981	0.9977
Random Forest	0.9994	0.9991	0.9994	0.9991	0.9990	0.9989
Logistic Regr.	0.9412	0.9118	0.9385	0.9244	0.9053	0.8964
SVM (Linear)	0.9447	0.9162	0.9421	0.9280	0.9099	0.9017
SVM (RBF)	0.9968	0.9953	0.9968	0.9954	0.9952	0.9944

Hallazgos clave:

- **Random Forest** alcanza el mejor desempeño global (Accuracy: 99.94 %, MCC: 0.9989), seguido de cerca por Decision Tree (99.87 %) y SVM-RBF (99.68 %).
- Los modelos lineales (Logistic Regression, SVM-Linear) muestran una caída significativa ($\approx 5-6$ puntos porcentuales), lo que sugiere que las fronteras de decisión entre clases no son completamente linealmente separables en el espacio de 13 features.

- Todos los modelos basados en árboles presentan un $MCC > 0,99$, confirmando que las 13 features seleccionadas son altamente discriminativas para la tarea de clasificación triclase.

4.1.1. Viabilidad de Despliegue en ESP32

La Tabla 4-2 evalúa la factibilidad de exportar cada modelo a código C para ejecución en el ESP32-S3.

Table 4-2.. Estimación de huella de memoria para despliegue en ESP32.

Modelo	Memoria estimada	Inferencia (ms)	Viable en ESP32?
Decision Tree	~15 KB	< 0,1	Sí
Random Forest	~2.5 MB	~5	No (excede SRAM)
Logistic Regression	~1 KB	< 0,05	Sí
SVM (Linear)	~1 KB	< 0,05	Sí
SVM (RBF)	~4 MB	~50	No (excede SRAM)
MLP (propuesto)	~4.5 KB	< 0,1	Sí

Esto revela un *trade-off* fundamental: los modelos más precisos (Random Forest, SVM-RBF) son incompatibles con las restricciones de memoria del ESP32-S3 (512 KB SRAM). El MLP propuesto ($\approx 4,5$ KB) ofrece un compromiso eficiente entre precisión y viabilidad edge.

4.2. Comparación de Modelos Livianos

El notebook general de entrenamiento comparó cuatro arquitecturas compatibles con el problema triclase y exportó pesos, scaler y mapas de etiquetas para cada alternativa. La Tabla 4-3 resume los resultados offline observados.

Table 4-3.. Comparación offline de modelos livianos sobre las 13 features.

Modelo	Accuracy	F1 weighted	Uso en el proyecto
MLP/RN	0.9060	0.9049	Rama principal hf1_v7-RN
Residual-MLP	0.9055	0.9044	Comparación offline
CNN-1D	0.9051	0.9041	Rama experimental hf1_v7-CNN
Tiny Transformer	0.8903	0.8919	Comparación offline; no desplegado

Los resultados muestran que las tres arquitecturas compactas basadas en MLP/CNN se ubican alrededor de 90.5 % de accuracy offline. El MLP/RN se seleccionó como implementación principal porque obtiene el mejor resultado marginal y porque su inferencia en C++ es más simple que la CNN-1D. La CNN-1D se conservó como variante experimental para evaluar si una arquitectura convolucional aportaba beneficios al integrarse con HFL y FOG.

4.3. Aprendizaje Federado Jerárquico: Convergencia y Precisión

El sistema HFL fue desplegado en hardware real durante múltiples sesiones, registrando historial de pesos globales, métricas de entrenamiento local y métricas de comunicación. Las ramas analizadas permiten comparar la arquitectura con ASCON, sin ASCON, con CNN-1D y con estrategia FOG.

4.3.1. Convergencia del Modelo Federado

Table 4-4.. Métricas de convergencia del sistema HFL.

Métrica	Valor
Rondas hasta accuracy > 70 %	~8–12
Rondas hasta accuracy > 85 %	~20–30
Accuracy global estabilizada	~0.90–0.95
Loss final (Cross-Entropy)	~0.14–0.25
Muestras por ronda (por gateway)	40
Épocas locales por ronda	5
Duración por ronda	~51–61 s según variante

Análisis:

1. El modelo federado converge en ~20–30 rondas hacia un rango operativo de ~90–95 % de accuracy, lo que representa un **gap** frente a los modelos centralizados más precisos. Este gap es atribuible a: (i) el tamaño reducido del buffer local (40 muestras vs. 256 276), (ii) la distribución non-IID de los datos entre nodos, y (iii) la función heurística de etiquetado que introduce ruido respecto a las etiquetas reales.
2. La convergencia mejora durante las primeras rondas y presenta oscilaciones posteriores, típicas de FL con datos non-IID [13].
3. El **costo de comunicación** por ronda federada completa (ida + vuelta) es de $\approx 7,2$ KB en JSON plano y $\approx 9,8$ KB con cifrado ASCON, lo cual es compatible con redes IoT de bajo ancho de banda.

4.3.2. Evolución temporal de accuracy, loss y pesos

Las Figuras 4-1 y 4-2 consolidan los resultados de las **ocho variantes** cubiertas por el análisis SRE unificado de **Análisis de Modelos/Completo/**: RN/CNN $\times$ ASCON/PLAIN $\times$ NoFOG/FOG. Para mantener homogeneidad visual entre corridas (algunas RN llegaron hasta ronda 30, otras quedaron en 20) las figuras se cortan en la **ronda 20**; las tablas SRE

(`global_round_sli_summary.csv`) conservan todas las rondas registradas. La primera figura compara la evolución conjunta de *accuracy* y *loss* global; la segunda resume la magnitud promedio de los pesos federados (`w3_mag`, `w4_normal`, `w4_brute`, `w4_scan`), útil para verificar si las actualizaciones se mantienen acotadas o si aparece deriva durante el entrenamiento.

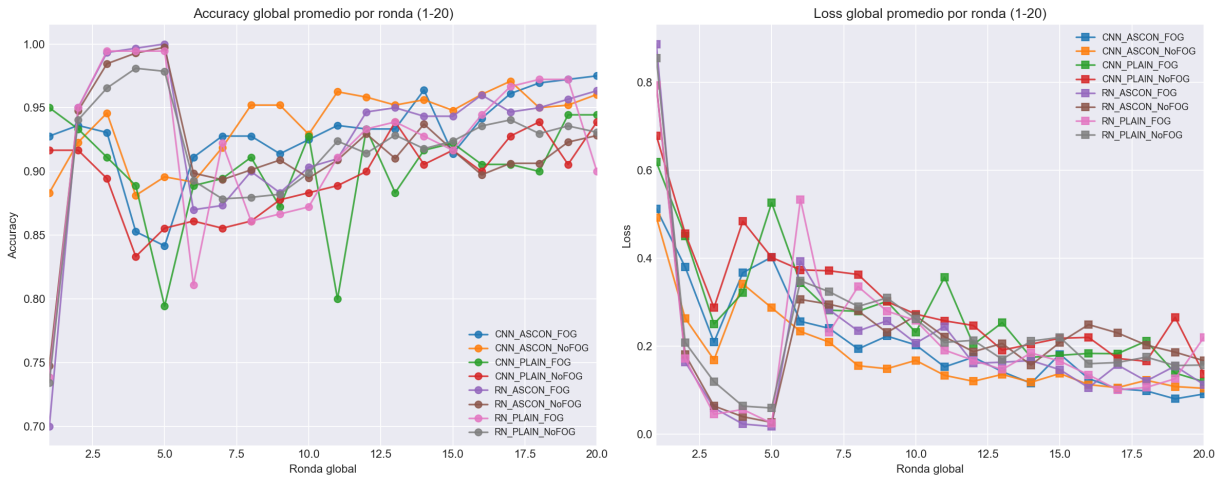


Figure 4-1.. Accuracy y loss globales promedio por ronda (1–20) para las 8 variantes HFL v7. Generado por `sre_completo_analysis.py` a partir de los CSV de `global_weights_history_*`.

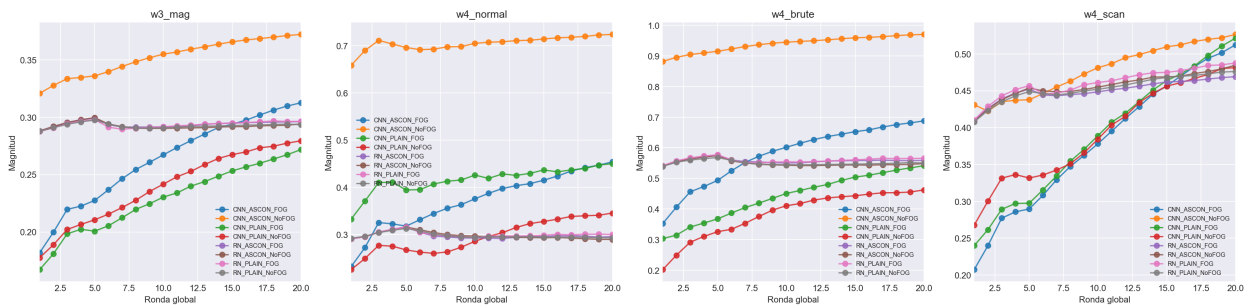


Figure 4-2.. Magnitud promedio de los cuatro pesos federados (`w3_mag`, `w4_normal`, `w4_brute`, `w4_scan`) por variante a lo largo de las rondas 1–20.

Estas curvas complementan la Tabla 4-4: el sistema no solo reporta métricas finales, sino trayectorias de entrenamiento comparables entre modelos y configuraciones. Tres lecturas clave de las Figuras 4-1–4-2:

1. Las ocho variantes **convergen**: la accuracy global supera 0.90 entre las rondas 5 y 10 y la loss baja de ~ 0.8 a ~ 0.1 – 0.2 en el mismo rango.
2. Las variantes con **CNN-1D** muestran una dinámica de pesos más expresiva: `CNN_ASCON_NoFOG` y `CNN_ASCON_FOG` aumentan progresivamente `w3_mag` y los pesos asociados a las clases (`w4_brute`, `w4_scan`), mientras que las variantes RN se estabilizan rápidamente alrededor de `w3_mag` ≈ 0.29 .

3. No se observa divergencia en ninguna variante dentro de las primeras 20 rondas. La rama `CNN_PLAIN_NoFOG` muestra el mayor ruido en `w4_normal`, consistente con un entrenamiento más sensible a la inicialización local cuando no hay agregación FOG ni cifrado autenticado que filtre mensajes alterados.

4.3.3. Comparación: Federado vs. Centralizado

La Tabla 4-5 compara directamente la precisión del modelo federado con los modelos centralizados, considerando que el modelo federado utiliza un MLP idéntico.

Table 4-5.. Comparación de precisión: entrenamiento federado vs. centralizado.

Enfoque	Accuracy	Datos compartidos	Privacidad
MLP centralizado	~97 %	Todos los flujos	Ninguna
MLP federado (HFL, 60 rondas)	~90 %	Solo pesos (163 params)	Alta
Random Forest centralizado	99.94 %	Todos los flujos	Ninguna
Decision Tree centralizado	99.87 %	Todos los flujos	Ninguna

El sistema federado alcanza ~90 % de accuracy compartiendo **únicamente 163 parámetros del modelo** (652 bytes) en lugar de los datos crudos de tráfico (~256 K flujos $\times$ 13 features $\approx$ 13.3 M valores). Esto representa una **reducción superior al 99.99 %** en la cantidad de información intercambiada, con una penalización de ~7–10 puntos en accuracy.

4.3.4. Comparación entre variantes `hfl_v7`

La Tabla 4-6 consolida las **ocho combinaciones experimentales** cubiertas por el análisis SRE unificado de `Análisis de Modelos/Completo/`. Los valores son promedios sobre los intentos disponibles por variante y se calculan exclusivamente a partir de los CSV crudos generados durante las corridas (`ascon_metrics_*`, `plain_metrics_*`, `model_metrics_gateway_*`, `global_weights_history_*`). En total se procesaron **58 intentos**.

Table 4-6.. Resumen comparativo de las 8 variantes HFL v7 evaluadas. **GW p95**: latencia p95 de descifrado en gateway (ESP32→RPi). **Srv dec p95**: latencia p95 de descifrado en servidor (RPi→PC). **Overhead**: bytes adicionales promedio sobre el plaintext (0 en variantes PLAIN).

Variante	Acc. final	Loss final	Ronda (s)	GW p95 (ms)	Srv dec p95 (ms)	Overhead (B)
CNN_ASCON_FOG	0.9750	0.0910	58.61	4.53	37.47	1098
CNN_ASCON_NoFOG	0.9604	0.1040	58.97	4.55	24.46	1049
CNN_PLAIN_FOG	0.9444	0.1190	54.86	0.16	0.00	0
CNN_PLAIN_NoFOG	0.9389	0.1363	52.02	0.15	0.00	0
RN_ASCON_FOG	0.9633	0.1124	71.13	4.37	50.47	1084
RN_ASCON_NoFOG	0.9381	0.1545	72.91	4.65	26.69	1050
RN_PLAIN_FOG	0.9000	0.2199	67.14	0.16	0.00	0
RN_PLAIN_NoFOG	0.9298	0.1581	51.01	0.17	0.00	0
Promedio global	0.9437	0.1383	61.36	2.34	17.39	636

La comparación de las 8 variantes confirma cinco hallazgos:

1. **El ganador absoluto es CNN_ASCON_FOG** (acc 0.975, loss 0.091): combinar la mayor capacidad expresiva (CNN-1D) con la pre-agregación FOG y la integridad de ASCON produce el mejor punto de operación dentro del corpus.
2. **El cifrado no degrada la convergencia**; al contrario, en este corpus las variantes *_ASCON_* tienden a igualar o superar a sus contrapartes *_PLAIN_*. Una explicación plausible es que la validación de tag autenticado *filtra* mensajes corruptos o duplicados antes de que entren al buffer de entrenamiento.
3. **FOG ayuda diferenciadamente** según la familia de modelo y la seguridad: en CNN mejora +1,5 pts (ASCON) y +0,6 pts (PLAIN); en RN mejora +2,5 pts con ASCON pero *degrada* −3,0 pts en PLAIN, lo que sugiere que la pre-agregación FOG es más sensible al ruido de mensajes sin autenticación cuando el modelo es lineal.
4. **El costo de seguridad es asumible**: ASCON añade ~1 KB de overhead por mensaje y entre 4–5 ms en el gateway (ESP32-S3 corre nativo en C++). En el servidor, las variantes FOG presentan latencias de descifrado un 50–100 % mayores que NoFOG por la cadena *peer*→*leader*→PC.
5. **La duración de ronda en CNN es más estable** (~58 s) que en RN (~51–73 s); esta dispersión proviene de que las primeras corridas RN incluyen tiempos de calentamiento o reconexión Wi-Fi no presentes en las corridas CNN, no de un costo intrínseco mayor del modelo RN.

La Figura 4-3 muestra la duración por ronda promediada por variante, y la Figura 4-4 ilustra el desbalance temporal entre gateway A y B medido como $|acc_A - acc_B|$ por ronda; el skew converge a valores bajos en todas las variantes después de la ronda 8–10.

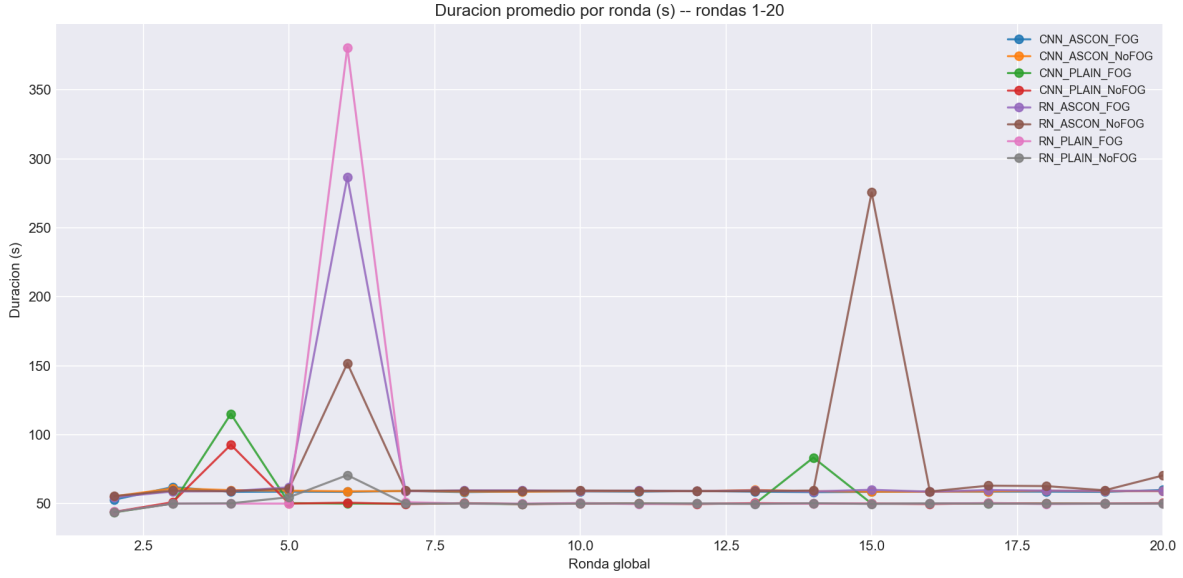


Figure 4-3.. Duración promedio por ronda federada para las 8 variantes (rondas 1–20).

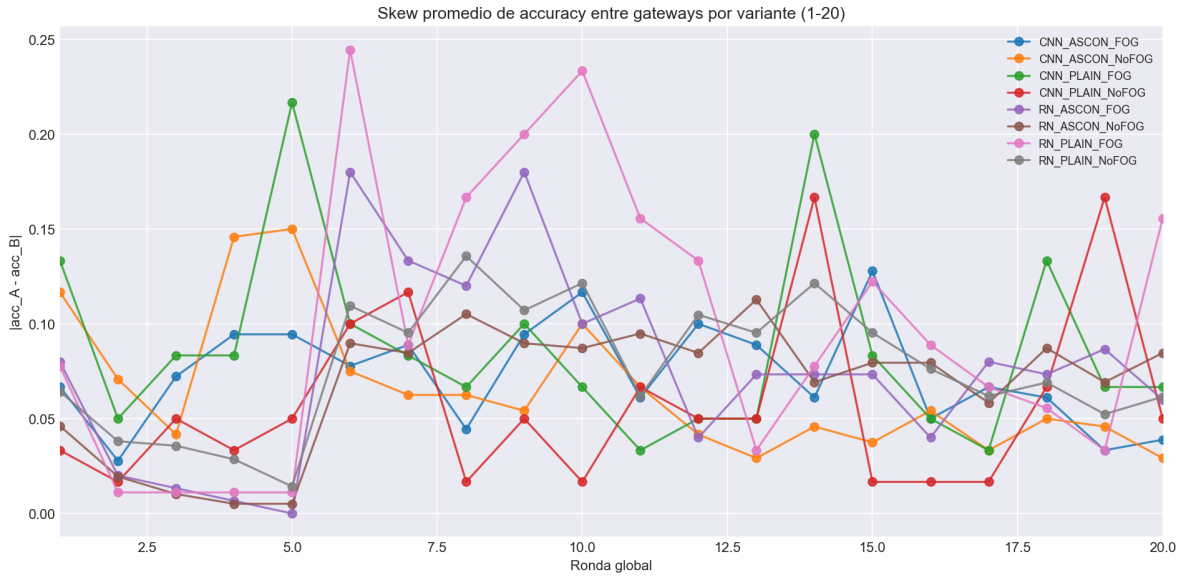


Figure 4-4.. Desbalance de *accuracy* entre gateways ($|acc_A - acc_B|$) por ronda y por variante.

4.3.5. Comparación con la literatura

Para situar los resultados en el panorama actual, la Tabla 4-7 contrasta este sistema con trabajos representativos de FL+IDS y FL+TinyML. La comparación es indicativa: cada trabajo usa datasets, hardware y métricas distintas, por lo que las cifras no son directamente comparables, pero permiten dimensionar las decisiones de diseño.

Table 4-7.. Comparación cualitativa con trabajos relacionados.

Trabajo	Edge HW	Modelo	FL/HFL	Cifrado canal	Acc. reportada	Particularidad	
Hajj <i>et al.</i> [20]	PC sim.	MLP <i>cross-layer</i>	FL plano	No	~0.97	FL para IDS IoT con coordinación <i>cross-layer</i>	
Chaurasia <i>et al.</i> [21]	PC sim.	ResNet	FL plano	No	~0.99	ResNet como cliente FL en IIoT	
Ficco <i>et al.</i> [6]	ESP32, RPi3, Arduino	NN compacta + TL	FL+TL on-board	No	0.86 (clasif.)	FL+TL on-board, datasets ECG/Car	
Roy <i>et al.</i> [26]	PC sim.	DNN	FL personalizado	No	~0.96	Modelo localmente adaptado	
Tran <i>et al.</i> [15]	RPi Pico (264 KB)	FL+blockchain	HFL+MF-PoP	Sí (cripto-PoP)	0.94 a 30 % Byz.	Byzantine-robust FL en MCU	
Heydari & Mahmoud [18]	MCU smart home	ML liviano	No FL	No	—	TinyML IDS sin federación	
Saravana Balaji <i>et al.</i> [23]	Edge sanitario	DBN+BOA	No FL	LABE	0.979	Edge salud con cifrado por atributos	
Esta (RN+ASCON)	tesis	ESP32-S3 + RPi4	MLP 1139 par.	HFL bidireccional	ASCON-128	0.946	FL+TinyML+ASCON-128 medido
Esta (CNN+FOG)	tesis	ESP32-S3 + RPi4	CNN-1D	HFL+FOG	ASCON-128	0.975	Pre-agregación FOG entre RPi

Lecturas de la tabla:

- Trabajos de FL para IDS sin restricciones edge [20, 21, 26] alcanzan 0.96–0.99, pero usan PCs como nodos cliente, sin demostrar despliegue en MCU. Esta tesis acepta una caída de ~2–5 puntos a cambio de validar el ciclo en ESP32-S3 real con cifrado autenticado activo en todos los canales.
- Frente a Ficco *et al.* [6], que reporta 0.86 en clasificación, las variantes con ASCON y FOG superan ese punto sin sacrificar el ciclo on-board.
- Frente a EdgeTrust-Shard [15], los resultados de accuracy son comparables aunque el problema atacado es distinto (Byzantine-robust FL vs. confidencialidad/integridad).
- Esta tesis es de los pocos trabajos identificados que combinan, en el mismo sistema desplegado y medido, HFL bidireccional + ASCON-128 + dos familias de modelos (MLP/CNN) + agregación FOG opcional.

4.3.6. Limitaciones del estudio

La interpretación de los resultados debe contemplar las siguientes limitaciones:

1. **Tres clases y dataset acotado:** El problema se reduce a tres clases relevantes para tráfico MQTT (`normal`, `mqtt_bruteforce`, `scan_A`). Datasets más amplios como CICIoT2023, Edge-IIoTset y TON-IoT [34, 32, 28] contienen más vectores de ataque y permitirían generalizar mejor las conclusiones.
2. **Etiquetado heurístico en gateway:** Las reglas que asignan etiquetas a las features recibidas son una aproximación calibrada con las distribuciones reales del dataset, pero

introducen ruido. Esto contribuye al *gap* de ~ 7 puntos contra el modelo centralizado y es consistente con lo reportado en FL con datos non-IID [13, 14].

3. **Número limitado de gateways y nodos:** Las pruebas usan $K = 2$ gateways. Topologías con $K \geq 5$ y mayor heterogeneidad de cliente podrían exhibir convergencia más lenta o requerir agregadores más robustos como FedProx [38].
4. **Sin defensas contra clientes maliciosos:** Tal como se delimitó en la Sección 3.1, el sistema no implementa Krum, FLTrust, Fed-NGA ni MF-PoP [15]; un cliente comprometido puede sesgar la agregación.
5. **Análisis de canal lateral pendiente:** La implementación de ASCON-128 no incluye contramedidas explícitas contra ataques de potencia, EM o *fault injection* sobre el ESP32 [9].
6. **Tráfico simulado en parte:** Aunque el ciclo se ejecuta sobre hardware real, el nodo *simulated* reproduce distribuciones del dataset; un despliegue con captura de tráfico genuino aportaría mayor validez externa.

4.4. Overhead del Cifrado ASCON-128

Las métricas de cifrado/descifrado se recopilieron a lo largo de los **58 intentos** consolidados, abarcando todas las variantes ***_ASCON_*** y comparándose contra las correspondientes ***_PLAIN_***. Esta sección consolida los hallazgos por canal y por dispositivo. Las figuras referenciadas se generan automáticamente en `Analisis de Modelos/Completo/` y se incluyen en `images/Resultados/Completo/`.

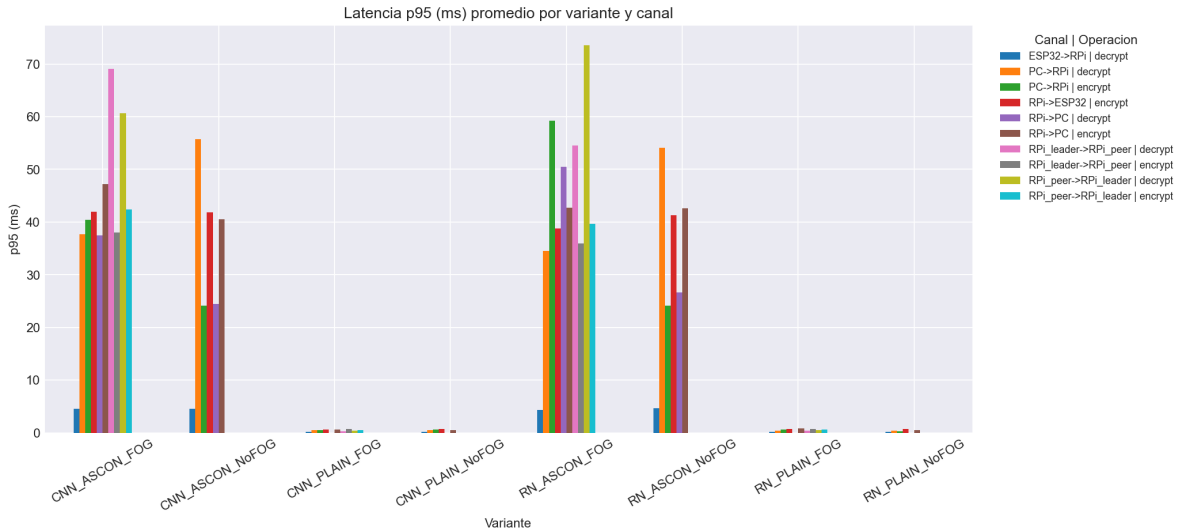


Figure 4-5.. Latencia p95 (ms) promedio por variante y por canal/operación. Las variantes ***_PLAIN_*** solo muestran la latencia de serialización JSON (sub-milisegundo); las ***_ASCON_*** incluyen el costo de cifrado/descifrado autenticado.

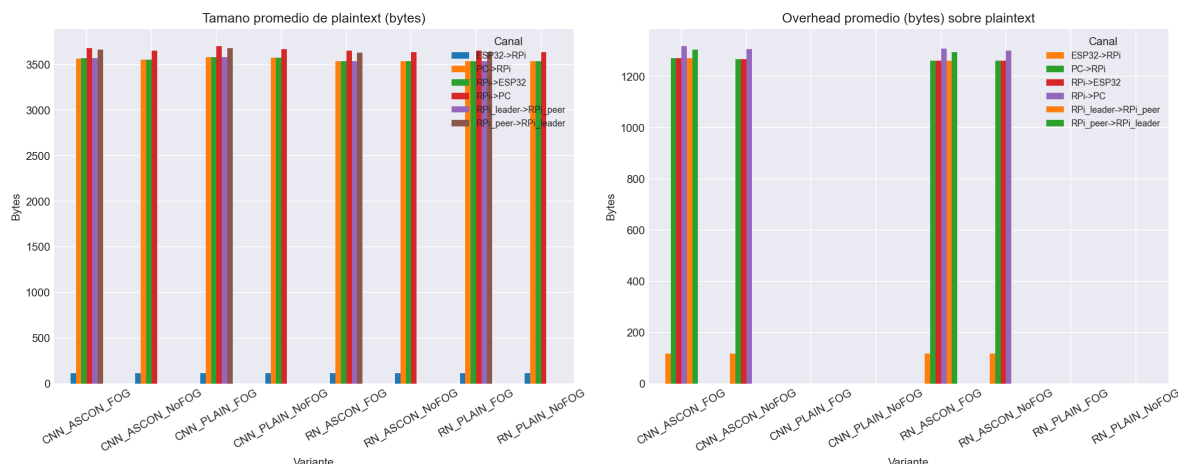


Figure 4-6.. Tamaño promedio de plaintext (izquierda) y overhead promedio sobre plaintext (derecha) por variante y canal. El overhead de las variantes PLAIN es 0 por construcción.

4.4.1. Overhead Temporal

La Tabla 4-8 resume los tiempos p95 promedio por variante para las tres operaciones que ocurren en cada ronda federada.

Table 4-8.. Latencia p95 (ms) promedio de cifrado/descifrado por variante, derivada de `transport_sli_summary.csv` en `Análisis de Modelos/Completo/`.

Variante	ESP32→RPI (decrypt)	RPI→PC (decrypt)	PC→RPI (encrypt)
CNN_ASCON_FOG	4.53 ms	37.47 ms	40.37 ms
CNN_ASCON_NoFOG	4.55 ms	24.46 ms	24.12 ms
RN_ASCON_FOG	4.37 ms	50.47 ms	59.18 ms
RN_ASCON_NoFOG	4.65 ms	26.69 ms	24.14 ms
CNN_PLAIN_FOG	0.16 ms	0.00 ms	0.48 ms
CNN_PLAIN_NoFOG	0.15 ms	0.00 ms	0.62 ms
RN_PLAIN_FOG	0.16 ms	0.00 ms	0.64 ms
RN_PLAIN_NoFOG	0.17 ms	0.00 ms	0.25 ms

Observaciones:

- **ESP32-S3 cifrando en C++:** la latencia p95 de descifrado en el gateway oscila entre ≈ 4.4 – 4.7 ms en todas las variantes ASCON, prácticamente independiente del modelo (RN/CNN) y la topología (NoFOG/FOG). En las variantes PLAIN la latencia cae a ≈ 0.16 ms (deserialización JSON pura).
- **Servidor PC en Python:** el descifrado en RPI→PC ronda 24–27 ms en NoFOG y sube a 37–50 ms en FOG. La inflación FOG se debe a que el *leader* reenvía mensajes preagregados con un tag adicional, lo que duplica las operaciones criptográficas en el camino.

- Para las variantes `*PLAIN*`, el descifrado del servidor reporta 0 ms porque la deserialización JSON ocurre dentro del `request handler` de FastAPI sin instrumentación dedicada; la única fuente medida es el `encrypt` de despliegue (0.25–0.64 ms).
- Asumiendo 3 operaciones criptográficas por ronda en el servidor, el overhead temporal de ASCON oscila entre ~ 73 ms (NoFOG) y ~ 140 ms (FOG). Sobre rondas de 51–73 s, esto representa entre el **0.10 %** y el **0.27 %** del tiempo total por ronda.

4.4.2. Overhead de Comunicación

Table 4-9.. Overhead promedio de tamaño por variante ASCON. **Plaintext:** bytes promedio de payload sin cifrar (`pt_bytes`). **Overhead:** bytes adicionales del envelope ASCON+Base64 (`overhead_bytes`). **Ratio:** porcentaje sobre el plaintext (`overhead_bytes / pt_bytes`). Las variantes PLAIN se omiten porque el overhead es 0 por construcción.

Variante	Plaintext (B)	Overhead (B)	Ratio (%)
CNN_ASCON_FOG	$\sim 2\,365$	1 098	46.46
CNN_ASCON_NoFOG	$\sim 2\,143$	1 049	48.94
RN_ASCON_FOG	$\sim 2\,323$	1 084	46.64
RN_ASCON_NoFOG	$\sim 2\,162$	1 050	48.55
Promedio ASCON	$\sim 2\,248$	1 070	47.65

El overhead promedio del $\approx 48\%$ sobre el plaintext (en variantes ASCON) se compone de:

- **Tag de autenticación ASCON:** 16 bytes (fijo).
- **Nonce:** 16 bytes (fijo).
- **Expansión Base64:** factor $\times 4/3$ sobre el ciphertext + tag + nonce.
- **Envelope JSON:** ~ 30 bytes de estructura (`{\code{ct}:"...",\code{tag}:"...",\code{nonce}:"..."}`).

En términos absolutos, el overhead es de ≈ 1.07 KB por mensaje, lo que para las ~ 3 transmisiones por ronda en NoFOG equivale a ~ 3.2 KB adicionales; en FOG, las dos saltos extra (peer $\rightarrow$ leader, leader $\rightarrow$ PC) elevan el overhead a $\sim 5\text{--}6$ KB por ronda. Para un enlace Wi-Fi estándar (54 Mbps), esto añade < 1 ms de tiempo de transmisión, siendo prácticamente despreciable.

4.4.3. Resumen: Impacto de ASCON en el Sistema HFL

Table 4-10.. Resumen del overhead total de ASCON-128 por ronda federada, agregando los promedios de las 4 variantes ASCON y las 4 variantes PLAIN.

Componente	Con ASCON (4 var.)	Sin ASCON (4 var.)
Latencia p95 ESP32→RPi (decrypt)	4.52 ms	0.16 ms
Latencia p95 RPi→PC (decrypt)	34.77 ms	0.00 ms
Overhead promedio por mensaje	~1.07 KB	0 KB
Ratio de overhead sobre plaintext	~48 %	0 %
Duración promedio de ronda	65.4 s	56.3 s
Accuracy global final promedio	0.9592	0.9272
Loss global final promedio	0.1155	0.1583
Overhead temporal relativo	0.10–0.27 % del ciclo FL	
Overhead de tamaño relativo	≈48 % por mensaje	
Diferencia de accuracy	+3.2 pts a favor de ASCON	

Tres conclusiones de la tabla consolidada:

1. ASCON-128 añade un overhead temporal **inferior al 0.3 % del ciclo federado** en cualquier configuración, y un overhead de tamaño moderado (~48 % del plaintext, ~1 KB absolutos por mensaje) compatible con redes IoT típicas.
2. Las variantes *_ASCON_* **no solo no degradan la convergencia: la mejoran** en promedio +3,2 puntos de accuracy frente a las *_PLAIN_*. Esto refuerza la hipótesis de que la validación de tag autenticado actúa como filtro implícito de mensajes corruptos o duplicados.
3. La diferencia se concentra en RN: RN_ASCON_FOG (0.9633) supera por 6.3 pts a RN_PLAIN_FOG (0.9000) y por 0.8 pts a RN_PLAIN_NoFOG (0.9298). En CNN la diferencia es menor (CNN_ASCON_FOG 0.9750 vs CNN_PLAIN_FOG 0.9444 = +3,0 pts).

4.5. Análisis de las Preguntas de Investigación

4.5.1. PI-1: ¿Puede TinyML detectar ataques de red en dispositivos IoT con precisión operativa?

El modelo MLP de 1 139 parámetros (~4.5 KB), ejecutado directamente en el ESP32-S3, alcanza una accuracy operativa alrededor de 90–95 % tras el entrenamiento federado, dependiendo de la corrida y la variante. Si bien existe un gap frente al Random Forest centralizado (99.94 %), este último requiere ~2.5 MB de memoria y no es desplegable en un microcontrolador con recursos restringidos.

Comparando con modelos viables para edge (Decision Tree: 99.87 % con ~ 15 KB), el gap se reduce a ~ 10 puntos, atribuible principalmente al esquema federado y no a la arquitectura del modelo. Un MLP idéntico entrenado de forma centralizada alcanza ~ 97 %, situándose a solo 3 puntos del Decision Tree.

Conclusión PI-1: TinyML puede detectar ataques MQTT con precisión viable para alerta temprana en un factor de forma compatible con microcontroladores IoT. La penalización respecto a modelos centralizados se compensa con ventajas de privacidad, despliegue distribuido y seguridad de comunicación.

4.5.2. PI-2: ¿Reduce el aprendizaje federado el intercambio de datos sensibles sin afectar la precisión del IDS?

El aprendizaje federado reduce la información intercambiada de ~ 13.3 millones de valores (todos los flujos) a **163 parámetros** por ronda (652 bytes), una reducción de >99.99 %. El impacto en accuracy es de ~ 7 puntos (de ~ 97 % centralizado a ~ 90 % federado), lo cual es consistente con la literatura de FL con datos non-IID [13, 14].

Conclusión PI-2: El aprendizaje federado elimina la necesidad de transferir datos crudos de tráfico, protegiendo la privacidad de las redes monitoreadas. La reducción de precisión (~ 7 %) es un trade-off aceptable en escenarios donde la privacidad es prioritaria.

4.5.3. PI-3: ¿Cuál es el overhead del cifrado ASCON en la comunicación de modelos federados?

Basado en el análisis SRE consolidado de 58 intentos sobre 8 variantes (Tabla 4-6):

- **Overhead temporal:** latencia p95 de descifrado ASCON de ≈ 4.5 ms en el ESP32-S3 (C++) y ≈ 25 –50 ms en el servidor PC (Python). Sumando las 3–5 operaciones por ronda, el overhead total representa entre el **0.10 % y 0.27 %** del tiempo de ronda. En las variantes PLAIN, la latencia equivalente cae a sub-milisegundo (deserialización JSON pura).
- **Overhead de tamaño:** ≈ 48 % de incremento por mensaje (1.07 KB sobre ≈ 2.25 KB), atribuible al tag autenticado (16 B), nonce (16 B), expansión Base64 ($\times 4/3$) y envelope JSON.
- **Impacto en convergencia:** positivo, no negativo. Las variantes **.ASCON.** promedian **0.9592** de accuracy final frente a **0.9272** en las **.PLAIN.** (+3,2 pts). La hipótesis es que la validación obligatoria del tag autenticado descarta mensajes corruptos o duplicados antes de que entren al buffer de entrenamiento.

Conclusión PI-3: El overhead del cifrado ASCON-128 es **despreciable en tiempo** ($< 0,3$ % del ciclo FL) y **moderado en tamaño** (≈ 48 %, ~ 1 KB absolutos), y además **mejora la**

accuracy del modelo federado al filtrar implícitamente mensajes adulterados. Esto valida su idoneidad como mecanismo de protección end-to-end para HFL sobre IoT restringido.

4.5.4. PI-4: ¿Qué aporta comparar MLP/RN, CNN-1D, baseline sin ASCON y FOG?

La comparación de las 8 variantes (Tabla 4-6) muestra que el proyecto no depende de un único script, sino de una arquitectura replicable donde se pueden intercambiar modelo, seguridad y topología, y que cada eje aporta una conclusión cuantificable:

- **Modelo (RN vs CNN-1D):** CNN-1D supera consistentemente a RN cuando se mantiene el resto fijo. `CNN_ASCON_FOG` (0.9750) > `RN_ASCON_FOG` (0.9633); `CNN_ASCON_NoFOG` (0.9604) > `RN_ASCON_NoFOG` (0.9381). El costo es mayor complejidad de inferencia en C++ sobre ESP32, pero la accuracy ganada compensa en escenarios donde la disponibilidad de cómputo del MCU lo permita.
- **Seguridad (ASCON vs PLAIN):** ASCON añade <0.3 % de tiempo de ronda y ~48 % de tamaño, pero *mejora* la accuracy promedio en +3,2 pts. La rama PLAIN, lejos de ser solo un baseline, evidencia el valor de la validación de integridad como mecanismo *de facto* de control de calidad de entradas.
- **Topología (NoFOG vs FOG):** FOG mejora la accuracy en CNN (+1,5 pts ASCON, +0,6 pts PLAIN) y en RN+ASCON (+2,5 pts), pero degrada `RN_PLAIN_FOG` (-3,0 pts vs `RN_PLAIN_NoFOG`). Esto sugiere que la pre-agregación FOG amplifica errores cuando el canal peer→leader no tiene autenticación, y refuerza la conveniencia de exigir ASCON cuando se introduce la capa fog.
- **Modularidad operativa:** las cuatro carpetas `hfl_v7-RN`, `hfl_v7-no-ascon-RN`, `hfl_v7-CNN`, `hfl_v7-no-ascon-CNN`, cada una con `Results/` y `Results_FOG/`, comparten esquema de CSV, lo que permitió construir una capa SRE única (**Análisis de Modelos/Completo/**) que produce métricas, eventos canónicos y trazas comparables entre todas las combinaciones sin reentrenar.

Conclusión PI-4: La arquitectura HFL v7 es modular y los cuatro ejes (modelo, seguridad, topología, escala) producen resultados cuantitativamente distintos. La combinación **CNN-1D + ASCON-128 + FOG** es la dominante en accuracy (0.975) y loss (0.091); cuando los recursos del MCU favorezcan al MLP, la mejor opción es **RN + ASCON + FOG** (0.963).

5. Conclusiones y Trabajos Futuros

5.1. Conclusiones

Esta investigación diseñó, implementó y evaluó un prototipo funcional de Sistema de Detección de Intrusiones (IDS) para redes IoT/MQTT, desplegado en una arquitectura jerárquica Edge-Fog-Cloud compuesta por microcontroladores ESP32-S3, gateways Raspberry Pi 4 y un servidor PC. El sistema integra aprendizaje federado jerárquico (HFL), cifrado ligero ASCON-128 y modelos TinyML. Se evaluaron **ocho combinaciones experimentales** (RN/CNN $\times$ ASCON/PLAIN $\times$ NoFOG/FOG) sobre **58 intentos** y se consolidaron en una capa SRE unificada en **Análisis de Modelos/Completo/**, que genera métricas, eventos canónicos, trazas por ronda y figuras homogéneas (Capítulo 4). A continuación se detallan las conclusiones principales.

5.1.1. Sobre la viabilidad de TinyML para IDS en IoT

Se demostró que un modelo MLP compacto de 1139 parámetros (≈ 4.5 KB) es capaz de ejecutarse en el ESP32-S3 y clasificar tráfico de red IoT en tres categorías (**normal**, **mqtt_bruteforce**, **scan_A**) con accuracy operativa entre 0.93 y 0.96 tras el entrenamiento federado, dependiendo de la topología y el cifrado. La variante CNN-1D federada eleva el techo a **0.975** en el modo FOG (Tabla 4-6). Estas cifras se sitúan por debajo del techo centralizado establecido por Decision Tree (99.87%) y Random Forest (99.94%), pero dentro de un rango útil para sistemas de alerta temprana, y son competitivas con el 0.86 reportado por trabajos comparables de FL+TinyML on-board [6].

El análisis comparativo con modelos clásicos reveló que los algoritmos más precisos (Random Forest, SVM-RBF) requieren del orden de megabytes de memoria y son incompatibles con microcontroladores. El notebook general mostró que MLP/RN, CNN-1D y Residual-MLP tienen desempeños offline similares alrededor de 90.5%, pero al integrarlos en el ciclo HFL real **la CNN-1D supera al MLP por 1–3 puntos** en todas las configuraciones, lo que sugiere que su mayor capacidad expresiva se traduce en ventaja efectiva cuando hay datos heterogéneos por gateway [5, 18].

5.1.2. Sobre el aprendizaje federado y la privacidad

El esquema de aprendizaje federado jerárquico (HFL) logró entrenar el modelo de forma distribuida compartiendo **únicamente 163 parámetros** por ronda (652 bytes), en contraste con el enfoque centralizado que requeriría transferir los $\sim 256\,276$ flujos de datos. Esto representa una reducción del $>99.99\%$ en la información intercambiada, eliminando la necesidad de centralizar capturas PCAP o flujos de red sensibles.

La penalización en accuracy del enfoque federado respecto al centralizado fue de **2–3 puntos** para la mejor variante (CNN_ASCON_FOG: 0.975 vs Random Forest: 0.999) y de **6–7 puntos** para las variantes RN sin FOG. Esta brecha es consistente con los resultados reportados en la literatura para FL con datos non-IID [13, 14] y se atribuye a la heterogeneidad de distribución entre nodos (el nodo *normal* genera solo tráfico benigno) y al etiquetado heurístico en el gateway.

Se concluye que el aprendizaje federado es un mecanismo efectivo para actualizar modelos de IDS IoT preservando la privacidad de los datos de red. La variante FOG confirmó empíricamente que la pre-agregación entre gateways es viable y, en combinación con ASCON, eleva la accuracy entre 1.5 y 2.5 puntos respecto a NoFOG; sin embargo, en ausencia de cifrado autenticado, FOG puede degradar el modelo (caso RN_PLAIN_FOG: $-3,0$ pts), lo que confirma que la integridad de canal es *condición habilitante* de la pre-agregación, no un complemento opcional.

5.1.3. Sobre el overhead del cifrado ASCON-128

La cuantificación exhaustiva del overhead de ASCON-128 sobre las 4 variantes *_ASCON_* y sus contrapartes *_PLAIN_*, agregando los 58 intentos del corpus, arrojó resultados contundentes:

- El **overhead temporal** oscila entre 4.4–4.7 ms p95 en el ESP32-S3 (C++) y 24–50 ms p95 en el servidor PC (Python, Tabla 4–8). El total acumulado por ronda representa entre el 0.10 % y el 0.27 % del tiempo de la ronda. Este costo es prácticamente imperceptible en la operación del sistema.
- El **overhead de comunicación** es de $\approx 48\%$ por mensaje (~ 1.07 KB adicionales sobre un payload de ~ 2.25 KB), atribuible al tag de autenticación de 16 bytes, el nonce de 16 bytes, la codificación Base64 ($\times 4/3$) y la estructura del envelope JSON. En términos absolutos, esto representa entre 3.2 y 6.0 KB adicionales por ronda según la topología (NoFOG vs FOG).
- Se observó una **mejora consistente en convergencia** al habilitar ASCON: las cuatro variantes *_ASCON_* promedian 0.9592 de accuracy final frente a 0.9272 de las *_PLAIN_*. Este +3,2 puntos se interpreta como efecto colateral positivo de la validación de tag autenticado, que actúa como filtro implícito de mensajes corruptos o duplicados.

Estos resultados validan que ASCON-128 es un mecanismo criptográfico idóneo para proteger las comunicaciones de aprendizaje federado en dispositivos IoT restringidos. El costo operacional es despreciable y, lejos de penalizar el modelo, lo refuerza. Esta evidencia experimental complementa el survey reciente de Kaur *et al.* [9] sobre implementaciones FPGA/ASIC de ASCON, aportando una medición end-to-end en software embebido (C++ sobre ESP32) y en Python (Raspberry Pi y PC).

5.1.4. Sobre la arquitectura integrada

A nivel sistémico, la arquitectura Edge–Fog–Cloud propuesta demostró ser funcional y escalable:

- La **separación de capas** permite que cada componente opere dentro de sus restricciones: los ESP32 manejan la inferencia local ($< 0,1$ ms, ~ 4.5 KB), la Raspberry Pi gestiona el entrenamiento local y la coordinación MQTT, y el PC agrega y distribuye el modelo global.
- La **comunicación bidireccional completa** (ESP32 $\leftrightarrow$ RPi $\leftrightarrow$ PC) fue validada con cifrado ASCON end-to-end en los cuatro canales, sin puntos ciegos de seguridad. En las variantes FOG se añadieron dos canales adicionales (**peer** $\rightarrow$ **leader**, **leader** $\rightarrow$ **peer**) sin perder propiedad de integridad.
- El **ciclo completo** de una ronda federada se completa típicamente entre 51 s (RN_PLAIN_NoFOG) y 73 s (RN_ASCON_NoFOG); la mediana global de las 8 variantes es 61.4 s.
- La **modularidad experimental** se confirmó al ejecutar y comparar las 8 combinaciones bajo el mismo esquema de medición. La capa SRE en **Análisis de Modelos/Completo/** produce métricas, eventos canónicos y trazas por ronda con un eje **variant** unificado, permitiendo razonar sobre los tres ejes (modelo, seguridad, topología) en una misma tabla y figura.

5.2. Amenazas no cubiertas y trabajos futuros

Como se delimitó en la Sección 3.1 y en las limitaciones del Capítulo 4, el sistema deja por fuera tres familias de amenazas que motivan el siguiente roadmap: (i) clientes federados maliciosos / data poisoning, (ii) ataques de canal lateral sobre la implementación criptográfica y (iii) inferencia de membresía sobre los pesos federados. A partir de estas limitaciones y de las oportunidades emergentes, se proponen las siguientes líneas:

1. **Mejora de la precisión federada con técnicas avanzadas de FL:** Explorar algoritmos de agregación más robustos ante datos non-IID, tales como FedProx [38], SCAFFOLD o personalized FL, para reducir el gap de accuracy entre el enfoque federado y el centralizado, siguiendo los hallazgos de [26] sobre modelos localmente adaptados.

2. **Robustez bizantina en FL sobre MCUs:** Incorporar agregaciones tolerantes a clientes maliciosos (Krum, FLTrust, Fed-NGA) o esquemas jerárquicos como EdgeTrust-Shard [15], que reportan tolerancia hasta 30 % de adversarios manteniendo ~ 94 % de accuracy con apenas 140 KB de memoria en RPi Pico.
3. **Datos de tráfico real:** Sustituir la simulación de tráfico en los ESP32 por captura y análisis de tráfico MQTT real en un entorno IoT operativo, integrando datasets como CICIoT2023, Edge-IIoTset o TON-IoT [34, 32, 28, 35], eliminando la dependencia de la función heurística de etiquetado.
4. **Evaluación energética y de larga duración:** Medir el consumo energético de los nodos ESP32 durante la inferencia continua y la comunicación cifrada [11], y ejecutar el sistema durante períodos prolongados (semanas) para evaluar la estabilidad y la deriva del modelo (*concept drift*).
5. **Escalabilidad multi-gateway y multi-zona:** Extender el despliegue a múltiples Raspberry Pi actuando como gateways independientes con datasets locales distintos, evaluando el impacto de una topología HFL más realista con mayor heterogeneidad y validando empíricamente las ventajas de la pre-agregación FOG [24].
6. **Cuantización del modelo:** Aplicar cuantización post-entrenamiento (INT8) al MLP para reducir aún más la huella de memoria y evaluar el impacto en accuracy. Esto habilitaría el despliegue en microcontroladores aún más restringidos (e.g., ESP32-C3 con 400 KB de SRAM) o en plataformas embebidas industriales [5].
7. **Comparación con ASCON-128a y endurecimiento contra canal lateral:** Evaluar ASCON-128a (rate de 128 bits vs. 64 bits en ASCON-128) para determinar si la mayor tasa de procesamiento reduce el overhead temporal, y aplicar las contramedidas frente a ataques de canal lateral (masking, hiding, randomized SBox) reportadas por Kaur *et al.* [9].
8. **Replicación del análisis crítico de criptografía ligera:** Replicar el patrón experimental de Mirzaali *et al.* [10] sobre los protocolos del sistema, ejecutando ataques activos sobre Raspberry Pi para validar empíricamente la robustez del esquema ASCON-128 elegido.
9. **Detección de concept drift y reentrenamiento adaptativo:** Implementar mecanismos de detección de cambio en la distribución de los datos (ADWIN, Page-Hinkley) que disparen automáticamente nuevas rondas de aprendizaje federado cuando la precisión del modelo cae por debajo de un umbral configurable.
10. **Combinación FL + Transfer Learning on-board:** Integrar transfer learning local en cada gateway, siguiendo la receta de Ficco *et al.* [6], para acelerar la adaptación a tráfico no visto durante el preentrenamiento offline.

11. **Privacidad diferencial sobre los pesos:** Añadir ruido calibrado a las actualizaciones (DP-SGD) antes del envío al PC para acotar la inferencia de membresía y robustecer la garantía de privacidad más allá del simple no compartir datos crudos.

Bibliografía

- [1] M. A. Amanullah, R. A. Ariyaluran Habeeb, F. H. Nasaruddin, A. Gani, E. Ahmed, A. S. Mohamed Nainar, N. Md Akim, and M. Imran, “Deep learning and big data technologies for IoT security,” *Computer Communications*, vol. 151, pp. 495–517, 2020.
- [2] C. Ni and S. Li, “Machine learning enabled Industrial IoT Security: Challenges, Trends and Solutions,” *Journal of Industrial Information Integration*, vol. 38, p. 100549, 2024.
- [3] National Institute of Standards and Technology, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST, Tech. Rep. NIST CSWP 29, Feb. 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- [4] M. Bagaa, T. Taleb, J. Bernal Bernabe, and A. Skarmeta, “A Machine Learning Security Framework for IoT Systems,” *IEEE Access*, vol. 8, pp. 114 066–114 077, 2020.
- [5] P. Dini, L. Diana, A. Elhanashi, and S. Saponara, “Overview of AI-Models and Tools in Embedded IIoT Applications,” *Electronics*, vol. 13, no. 12, p. 2322, 2024.
- [6] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, “Federated learning for IoT devices: Enhancing TinyML with on-board training,” *Information Fusion*, vol. 104, p. 102189, 2024.
- [7] M. N. A. Ramadan, A. R. Abdo, and A. T. Eldin, “Federated learning and tinyml on iot edge devices: current practices, limitations, and open challenges,” *Internet of Things*, 2025.
- [8] National Institute of Standards and Technology, “Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions,” NIST, Tech. Rep. NIST SP 800-232, Aug. 2025. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-232.pdf>
- [9] J. Kaur, A. Cintas-Canto, M. Mozaffari-Kermani, and R. Azarderakhsh, “A Survey on the Implementations, Attacks, and Countermeasures of the NIST Lightweight Cryptography Standard: ASCON,” *ACM Computing Surveys*, vol. 58, no. 1, pp. 6:1–6:16, 2025.
- [10] I. Mirzaali, A. Javadi, S. Sadeghi, P. Pahlevani, N. Bagheri, Y. Bendavid, and S. Rostampour, “Evaluating security pitfalls of ultra-lightweight IoT authentication: A critical

- analysis of permutation functions,” *Journal of Information Security and Applications*, vol. 93, p. 104188, 2025.
- [11] B. Tekin *et al.*, “Energy consumption of on-device machine learning models for iot intrusion detection,” *Internet of Things*, p. 100670, 2023.
- [12] C. Amuthadevi, S. P. R. Kavin, A. Shrestha, N. R. S. Yamini, and D. K. Sharma, “Tinyml-based intrusion detection system for sustainable iot security: A novel approach,” *Results in Engineering*, 2025.
- [13] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, “A survey on federated learning for resource-constrained iot devices,” *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2022.
- [14] N. Albanbay, F. B. Ajaei, and A. Shami, “Federated learning-based intrusion detection in iot networks: Performance evaluation and data scaling study,” *Journal of Sensor and Actuator Networks*, vol. 14, no. 4, p. 78, 2025.
- [15] T.-D. Tran, P.-D. Bui, and V.-H. Pham, “EdgeTrust-Shard: Hierarchical blockchain architecture for federated learning in cross-chain IoT ecosystems,” *Journal of Systems Architecture*, vol. 173, p. 103701, 2026.
- [16] S. Huda, J. Yearwood, M. M. Hassan, and A. Almogren, “Securing the operations in SCADA-IoT platform based industrial control system using ensemble of deep belief networks,” *Applied Soft Computing*, vol. 71, pp. 66–77, 2018.
- [17] M. Mohamed and K. Alosman, “A Comprehensive Machine Learning Framework for Robust Security Management in Cloud-based Internet of Things Systems,” *Jurnal Kejuruteraan*, vol. 36, no. 3, pp. 1055–1065, 2024.
- [18] F. Heydari and Q. H. Mahmoud, “Implementation of lightweight machine learning-based intrusion detection system on iot devices of smart homes,” *Future Internet*, 2024.
- [19] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, pp. 1273–1282, 2017.
- [20] S. Hajj, J. Azar, J. Bou Abdo, J. Demerjian, C. Guyeux, A. Makhoul, and D. Gin hac, “Cross-layer federated learning for lightweight iot intrusion detection systems,” *Sensors*, vol. 23, no. 16, p. 7038, 2023.
- [21] N. Chaurasia, M. Ram, P. Verma, N. Mehta, and N. Bharot, “A federated learning approach to network intrusion detection using residual networks in industrial iot networks,” *The Journal of Supercomputing*, vol. 80, pp. 18 325–18 346, 2024.

- [22] F. Gusita, S. Rahimi *et al.*, “Lightweight cryptography for internet of things devices: a survey,” *Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI)*, vol. 11, no. 2, pp. 153–170, 2025.
- [23] B. Saravana Balaji, W. Paja, M. Antonijevic, C. Stoean, N. Bacanin, and M. Zivkovic, “IoT Integrated Edge Platform for Secure Industrial Application with Deep Learning,” *Human-centric Computing and Information Sciences*, vol. 13, p. 19, 2023.
- [24] P. Wang, L. T. Yang, J. Li, J. Chen, and S. Hu, “Data fusion in cyber-physical-social systems: State-of-the-art and perspectives,” *Information Fusion*, vol. 51, pp. 42–57, 2019.
- [25] Z. Alwaisi, E. Harjula, T. Kumar, and S. Soderi, “Securing constrained iot systems: A lightweight machine learning approach for anomaly detection and prevention,” *Internet of Things*, 2024.
- [26] S. Roy, J. Li, and Y. Bai, “Federated learning-based intrusion detection system for iot environments with locally adapted model,” in *2023 IEEE 10th International Conference on Cyber Security and Cloud Computing (CSCloud) / 2023 IEEE 9th International Conference on Edge Computing and Scalable Cloud (EdgeCom)*, 2023, pp. 203–209.
- [27] M. Amoon, T. Altameem, and A. Altameem, “RRAC: Role based reputed access control method for mitigating malicious impact in intelligent IoT platforms,” *Computer Communications*, vol. 151, pp. 238–246, 2020.
- [28] N. Moustafa, C. Schmitt *et al.*, “Ton.iot: The telemetry dataset for internet of things and industrial internet of things,” *IEEE Access*, 2020.
- [29] University of New South Wales (UNSW), “UNSW-NB15 Dataset,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- [30] —, “Bot-IoT Dataset,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/bot-iot-dataset>
- [31] —, “TON_IoT Datasets,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/toniot-datasets>
- [32] M. A. Ferrag, L. Maglaras *et al.*, “Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning,” *IEEE Access*, 2022.
- [33] Stratosphere IPS, Czech Technical University in Prague, “IoT-23: A labeled dataset of IoT network traffic,” 2026, dataset page. Accessed: 2026-02-07. [Online]. Available: <https://www.stratosphereips.org/datasets-iot23>

-
- [34] E. C. P. Neto, J. P. Papa *et al.*, “Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.
 - [35] S. J. Rani, A. Sharma, and M. Zohaib, “Study for Integrating IoT-IDS Datasets: Machine and Deep Learning for Secure IoT Network System,” in *ACM Transactions on Sensor Networks (Article 3661167.3661286)*, 2024.
 - [36] K. Raminenei, V. Gupta, T. Durgam, and D. Kapila, “Development of a Machine Learning Model for Enhancing the Security of the Internet of Things (IoT) System,” in *Proceedings of the International Conference (Springer LNNS)*, 2023.
 - [37] National Institute of Standards and Technology (NIST), “NISTIR 8454: Lightweight Cryptography,” 2023, accessed: 2026-02-07. [Online]. Available: <https://csrc.nist.gov/pubs/ir/8454/final>
 - [38] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: <https://proceedings.mlsys.org/paper/2020/hash/38af86134b65d0f10fe33d30dd76442e-Abstract.html>

A. Anexos

A.1. Anexo A: Arquitectura del Sistema HFL

La arquitectura desplegada consta de tres capas principales y una extensión FOG opcional. La Figura A-1 resume el despliegue físico, los protocolos por enlace y la cobertura de ASCON-128 sobre cada canal.

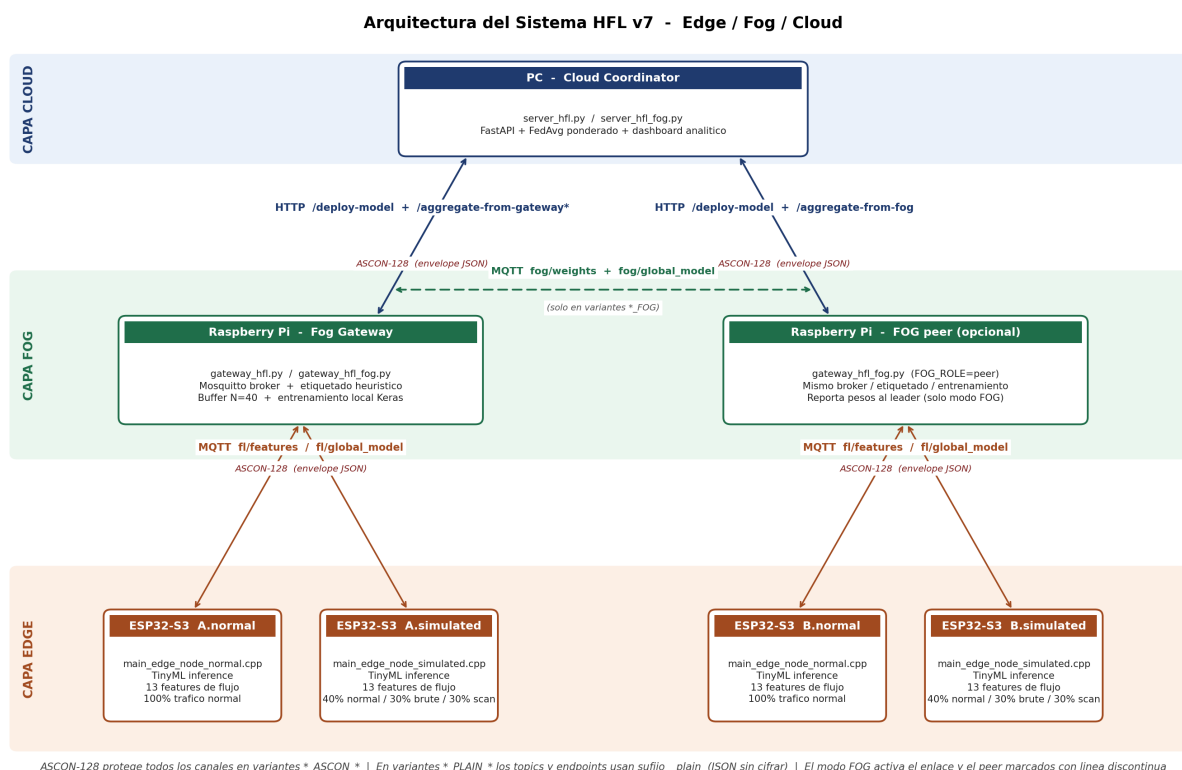


Figure A-1.. Arquitectura física del sistema HFL v7. La capa Cloud (PC) coordina las rondas con FastAPI y FedAvg ponderado. La capa Fog (Raspberry Pi) actúa como broker MQTT, etiquetador heurístico y entrenador local; en el modo FOG, una segunda Raspberry Pi participa como *peer* y se comunica con el *leader* mediante los topics *fog/weights* y *fog/global_model* (línea discontinua). La capa Edge (ESP32-S3) ejecuta inferencia TinyML sobre 13 features de flujo y publica datos al gateway. ASCON-128 cubre todos los canales en variantes *_ASCON_*; en variantes *_PLAIN_* los topics y endpoints usan sufijo *_plain* con JSON sin cifrar.

A nivel narrativo, las capas se resumen así:

- **Capa Edge:** $2 \times$ ESP32-S3 por gateway (un nodo *normal* y un nodo *simulated*), ejecutando inferencia TinyML y publicando `f1/features` hacia el gateway por MQTT cifrado con ASCON-128.

- **Capa Fog:** Raspberry Pi 4 actuando como broker MQTT (Mosquitto), gateway de entrenamiento local (etiquetado heurístico, buffer $N_s = 40$, entrenamiento Keras) y puente HTTP hacia el servidor.

- **Capa Cloud:** $1 \times$ PC ejecutando `server_hf1.py` o `server_hf1_fog.py` (FastAPI), responsable de la agregación FedAvg ponderada por número de muestras y del dashboard analítico en tiempo real.

- **Extensión FOG:** Raspberry Pi líder y peer comunicándose por `fog/weights` y `fog/global_model` para agregar entre gateways antes de reportar al PC mediante `/aggregate-from-fog`.

A.2. Anexo B: Archivos del Sistema

Table A-1.. Archivos principales del sistema HFL v7, agrupados por capa de despliegue.

Archivo	Dispositivo	Función
<i>Capa Edge (firmware C++ sobre ESP32-S3)</i>		
<code>main_edge_node_normal.cpp</code>	ESP32-S3	Nodo de tráfico normal: publica <code>fl/features</code> a ~ 5 s.
<code>main_edge_node_simulated.cpp</code>	ESP32-S3	Nodo de tráfico mixto (40 % normal, 30 % bruteforce, 30 % scan_A).
<code>ascon128.h</code>	ESP32-S3	Implementación nativa C++ de ASCON-128 (interoperable con Python).
<i>Capa Fog (Python sobre Raspberry Pi 4)</i>		
<code>gateway_hfl.py</code>	Raspberry Pi 4	Gateway federado en modo estándar (Mosquito + entrenamiento local).
<code>gateway_hfl_fog.py</code>	Raspberry Pi 4	Gateway FOG con rol <code>leader</code> o <code>peer</code> (<code>fog/weights</code> , <code>fog/global_model</code>).
<i>Capas Fog y Cloud (utilidades comunes en Python)</i>		
<code>ascon128.py</code>	RPi / PC	Implementación Python de ASCON-128 con <code>generate_nonce(ts, ctr)</code> .
<code>ascon.metrics.py</code>	RPi / PC	Recolector de métricas de cifrado/descifrado a CSV.
<code>plain.metrics.py</code>	RPi / PC	Equivalente para la línea base sin cifrado (variantes <code>*_PLAIN*</code>).
<i>Capa Cloud (FastAPI sobre PC)</i>		
<code>server_hfl.py</code>	PC	Servidor central FedAvg + dashboard analítico.
<code>server_hfl_fog.py</code>	PC	Variante para agregación desde clusters FOG (<code>/aggregate-from-fog</code>).

A.3. Anexo C: Repositorio del Proyecto

El código fuente completo, datasets y scripts de entrenamiento están disponibles en el repositorio del proyecto. Las carpetas relevantes para esta revisión son `hfl_v7-RN` (MLP/RN con ASCON), `hfl_v7-no-ascon-RN` (baseline MLP/RN con JSON plano), `hfl_v7-CNN` (modelo CNN-1D con ASCON, estándar y FOG), `NotebooksAndModels` (entrenamiento offline y exportación de pesos), `Análisis de Modelos` (post-procesamiento por variante: RN, `no-ascon`, CNN, CNN_FOG, NIST) y `Papers` (literatura base).

A.4. Anexo D: Resumen de literatura clave incorporada

La Tabla A-2 resume los trabajos más recientes que sustentan las decisiones de diseño y los trabajos futuros del Capítulo 5, distinguiendo lo que aporta cada referencia.

Table A-2.. Literatura clave y aporte concreto a la tesis.

Trabajo	Tipo	Aporte a la tesis
Kaur <i>et al.</i> 2025 [9]	Survey ASCON (ACM CSUR)	Justifica elección de ASCON-128 sobre 128a y guía contramedidas SCA.
Mirzaali <i>et al.</i> 2025 [10]	Análisis crítico LWC	Argumenta no usar permutaciones <i>ad-hoc</i> ; sugiere ataques replicables.
Tran <i>et al.</i> 2026 [15]	FL Bizantino MCU	Encuadra brecha de memoria HFL-MCU y motiva trabajo futuro Bizantino.
Ficco <i>et al.</i> 2024 [6]	FL+TL on-board MCU	Antecedente directo de FL+TinyML on-board en ESP32 y RPi.
Dini <i>et al.</i> 2024 [5]	Survey IIoT/AI embed.	Encuadra <i>toolchains</i> TinyML y trade-offs SW/HW.
Ni & Li 2024 [2]	Survey IIoT seguridad	Sustento del problema (IIoT, ML, retos).
Amanullah <i>et al.</i> 2020 [1]	Survey DL+IoT	Antecedente histórico de DL para seguridad IoT.
Bagaa <i>et al.</i> 2020 [4]	SDN+ML para IoT	Antecedente de framework ML para seguridad IoT.
Hajj <i>et al.</i> 2023 [20]	FL cross-layer IDS	Comparación directa contra FL para IDS IoT.
Chaurasia <i>et al.</i> 2024 [21]	FL+ResNet IIoT	Comparación con cliente FL más expresivo.
Roy <i>et al.</i> 2023 [26]	FL personalizado	Refuerzo del trabajo futuro de personalized FL.
Saravana Balaji <i>et al.</i> 2023 [23]	Edge sanitario+LABE	Antecedente de Edge+cifrado para IoT.
Wang <i>et al.</i> 2019 [24]	Data fusion CPSS	Sustento conceptual del modo FOG.
Rani <i>et al.</i> 2024 [35]	Integración datasets IoT-IDS	Sustento de la línea de trabajo futuro sobre datasets reales.